Fast to Write, Slow to Learn:

Instant N-gram Notes that Upgrade to Learning Neurons for Two-Layer Dynamic Memory in a Production Inference Engine

Ashley Hart and **Petrus Pennanen**

TensorFold.dev

Abstract (short)

We give an LLM a two-stage memory system that works while it serves, with all of the model's original weights kept frozen. TensorFold Notes store each fact as the model's exact token ids. A note is written instantly and deleted exactly from the store: the few notes a question needs go into its prompt. Living Weights teach a fact into a small pocket of extra neurons while the server keeps answering. We ran our generated tests with Nemotron 3.5 Lightning. On our test set with two-step questions, a few retrieved notes answer as well as putting every fact in the prompt. They use fewer tokens. And they answer far better than the taught neurons alone. Every night, the server teaches the neurons the most useful and durable facts from the notes. After a fact's note is taken out of the prompt, the taught neurons still answer it.

Abstract (full)

A language model in use keeps hearing facts it should remember, such as a new address or a changed preference. Retraining is too slow for this, and putting every fact into the prompt costs tokens on every answer.

DeepSeek's Engram showed that an n-gram lookup table inside a model is a cheap, high-capacity memory, but its rows are learned in pretraining and fixed afterwards. An earlier implementation 'User as Engram' writes per-user rows at run time, but only into a model pretrained with such tables.

We propose two memories for a model at run time. The lower one is TensorFold Notes. It is a dynamic n-gram memory on exact token ids. Its notes are written instantly, kept exactly and versioned. They are deleted exactly from the note store. The upper one is Living Weights. It is a pocket of extra feed-forward neurons beside each MoE layer's shared expert. The pocket is trained on the running server from a single sentence. It answers reworded questions that exact n-grams cannot match. Both memories keep the model's original weights frozen.

A mechanism connects these systems. It looks at the facts in use that stay unchanged and finds the most useful of them. Then it promotes them from notes into neurons. A change or take-back reaches the notes at once. It is used from the next request. The neurons are then

corrected the next night. We describe Living Weights as shipped in TensorFold for NVIDIA Nemotron 3.5 Lightning (30B-A3B). It runs on Apple Metal and NVIDIA CUDA. We evaluate it with our generated test suite. The right answers in the suite come from a rule book for changes of mind, take-backs and dates. The rule book is proved in Lean.

On our hardest test set, a few retrieved notes answer as many questions as giving the model every fact. They answer about twice as many as the trained neurons alone. On two-step questions they answer about five times as many. On a mixed set of 17 lessons, they answer more than the full prompt with less than half the prompt tokens. Once the neurons have learned a fact, its note can be taken out of the prompt. The neurons then answer alone. In our longest run this held for five facts. They were answered on 14 of 14 questions. An untaught copy of the model answered none.

We also report what the neurons cost to learn and to serve as measured on an NVIDIA GB10. Teaching a fact to neurons in a background process takes about three minutes, and a long prompt is read only about 2% slower.

Code

The TensorFold engine with Living Weights: github.com/ashhart/TensorFold

Dynamic n-gram memory (TensorFold Notes), with the tables and receipts behind Section 7: github.com/ThinkOffApp/dynamic-ngram-memory

1. Introduction

A deployed model meets three kinds of new facts. Some are about the user ("My sister is Anna."). Some change ("Pekka sold the Volvo; he drives a green Saab now."). Some are withdrawn ("I no longer like salty liquorice."). A useful memory has to do four things. It has to take all three at the moment they are said. It has to answer them later in any wording. It has to forget what was withdrawn. And it has to cost little on every answer. Our examples follow one small family through the paper. There is the user and their sister Anna. There is Anna's husband Pekka, with his green Saab. And there is aunt Liisa, who lives in Tampere, a city in Finland.

The usual answers each fail one of these. Fine-tuning is slow and edits everything at once. Retrieval into the prompt is exact and easy to delete. But its cost grows with every answer's prompt. And it matches the words of a question rather than its meaning. Model editors such as ROME and MEMIT write facts into existing weights, and degrade as edits accumulate [Meng 2022a, 2022b]. Side-memory editors keep the base frozen and add parameters that switch on only for their edit [Mitchell 2022; Huang 2023; Hartvigsen 2023; Wang 2024; Wang 2025]. They report that rewordings remain their weak spot.

DeepSeek's Engram [Cheng 2026] made a strong case for a third kind of memory. It is a large table of learned vectors, added inside a few of the model's layers. The table is addressed by hashing the last few tokens. User as Engram [Li 2026] writes such rows at run time. It reports 99.2% recall of single facts on models of 178M to 1.22B parameters. When

two facts must be chained, recall is 12.9%. The model we serve, Nemotron 3.5 Lightning, has no Engram table, and neither do most deployed models.

We propose to split run-time memory into two layers with different speeds, in the spirit of complementary learning systems [McClelland 1995]:

- **A fast, exact lower layer: TensorFold Notes.** Each fact is kept as its exact token ids with an index of its n-grams, a version, an owner and a source. A note is written the moment it is said, changed by adding a version, and withdrawn by a tombstone. Notes are many, cheap and exact, and they are retrieval, not learning.
- **A slow, generalising upper layer: Living Weights.** A pocket of extra neurons beside each MoE layer's shared expert. A fact takes about ten of them, trained on the running server in about three minutes. The neurons key on the model's hidden state, not on tokens, so they can answer a question whose words the fact never used.
- **Movement between them.** The facts that prove useful and stay unchanged are promoted into neurons. This happens nightly and with hysteresis. A note stays the record of truth until repeated regressions show the neurons answer without it. A change or take-back reaches the notes at once. It is used from the next request. The neurons are corrected at the next night.

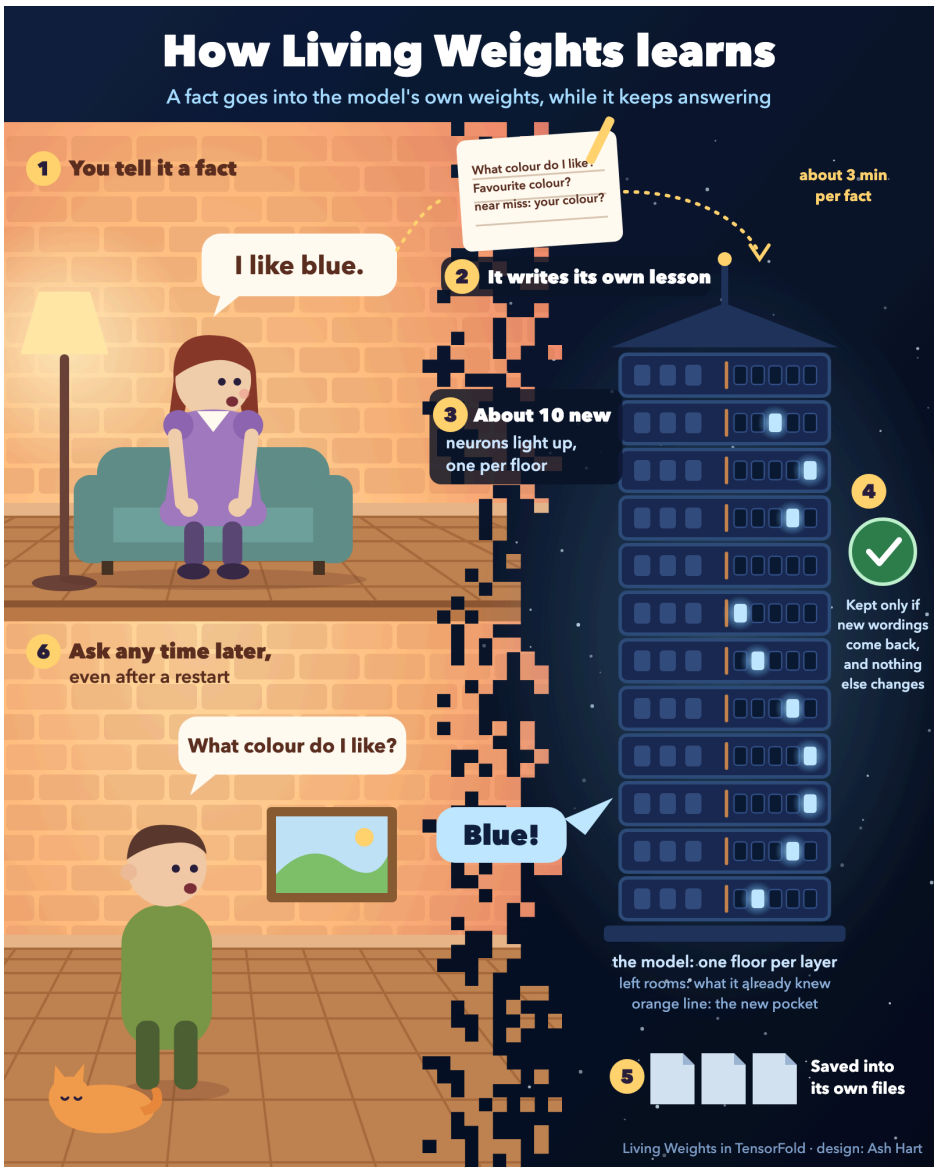


Figure 1. She says a fact. The model writes its own lesson, about ten floors light up, and the fact is kept only if the checks pass. Later a different question still gets the answer.

Indexing notes by their words is classic retrieval [Robertson 2009; Khandelwal 2020; Wu 2022]. What is new is doing it inside the serving engine. It is exact on token ids. At 1,000,000 notes a lookup takes a median of about 3 ms on one thread (Section 4.2). Notes are added live, and learned neurons sit beside them.

Contributions

1. **Living Weights (Section 3, TensorFold): run-time-trained neuron pockets in a 30B MoE model.** They are taught with sentences on the serving engine, with results kept only when questions are answered correctly. They are then saved into the checkpoint and served on Metal and CUDA. Measured cost on a GB10 and our solution to remove most of it are reported in Section 7.5.
2. **A generated, rule-checked test suite for run-time memory** (Section 6). It covers changes of mind, take-backs, dates, unknowns, yes/no and two-step questions. Its

rule book is proved in Lean, and it has two references: all lessons in the prompt, and an exact latest-value lookup.

3. **Measurements of the neuron layer** on 1 to 64 facts (Section 7), including two-step questions and a retention curve.
4. **The lower layer, TensorFold Notes, and the design of promotion** (Sections 4 and 5). The notes run inside the TensorFold server, and through the Notes Proxy in front of other inference engines. We give the first notes-only measurements (Sections 4.7 and 7.7). Promotion runs inside the server. In one run on a GPU, five facts left the prompt. They are answered by the neurons alone (Section 7.8).

2. Background and related work

N-gram memories. Cache language models raised the probability of recently seen words as a text went on [Kuhn 1990]. Infini-gram counts n-grams of any length over trillions of tokens with fast exact lookup, built offline [Liu 2024]. Engram adds hashed 2- and 3-gram embedding tables inside the model. The tables are learned in pretraining. The paper reports a 100B-parameter table held in host memory. It costs 1.9 to 2.8% of throughput on an H800 [Cheng 2026]. User as Engram makes such rows per-user and writable at run time. A write takes under 1 ms in closed form, or about 1 s with a few gradient steps. A fact takes about 16 rows. Independent writes keep about 0.98 top-1 up to 1,000 facts. One shared table falls from 68% to 35% top-1 between 100 and 1,000 facts [Li 2026]. EngramEdit edits shared n-gram embeddings of an Engram model and reports 2,000 sequential edits with 97.0 generalization [Cai 2026]. Conditional memory adapters add Engram-style memory to a frozen model after pretraining [When to Adapt 2026]. This is the closest published work to adding n-gram memory to a model that never had it. They target domain specialisation, not per-fact writes. Hashed n-gram rows are shared by unrelated phrases [TF-Engram 2026], and a writable memory table can carry a backdoor [BadEngram 2026].

Retrieval memories. kNN-LM interpolates the model's next-token distribution with nearest neighbours from a datastore keyed by hidden states [Khandelwal 2020]. Memorizing Transformers attend over a growing external memory of past keys and values [Wu 2022]. Lexical retrieval with BM25 [Robertson 2009] and retrieval-augmented generation put text into the prompt. Fine-tuning has been compared with retrieval for injecting knowledge, with retrieval ahead for new facts [Ovadia 2024].

Editors. ROME and MEMIT locate and rewrite factual associations in feed-forward weights [Meng 2022a, 2022b]. AlphaEdit constrains edits to a null space [Fang 2025]. Side-memory editors leave the base frozen. SERAC stores edits with a learned scope classifier [Mitchell 2022]. Transformer-Patcher adds one trainable neuron per mistake to the last feed-forward layer. It reports about 2,300 sequential edits at 1.4 to 4.5% model growth. It also reports that retention falls from 0.997 to 0.069 without a loss that keeps the neuron silent elsewhere [Huang 2023]. GRACE keeps a codebook of keys with radii at one layer [Hartvigsen 2023]. WISE keeps a side copy of a feed-forward layer. It reports routing, not storage, as the bottleneck [Wang 2024]. MEMoE uses mixture-of-experts adaptors [Wang 2024b]. MEMOIR applies an edit only when the input's pattern overlaps a stored one [Wang 2025]. Multi-hop questions after edits are hard for all of them [Zhong 2023]. Scoring by the model's own

written answer, rather than teacher-forced probabilities, lowers single-edit success from 96.8% to 38.5% in one study [Yang 2025].

Trained memory inside the model. Product-key memories [Lample 2019] and memory layers at scale [Berges 2024] add large sparse key-value layers trained with the model. Sparse Memory Finetuning updates only the slots a batch touches. It reports far less forgetting than full fine-tuning or LoRA [Lin 2025]. Feed-forward layers act as key-value memories [Geva 2021], which is the reading we use for pocket neurons.

Where Living Weights sits. The pocket is a side-memory editor. Its closest ancestor is Transformer-Patcher. It differs in five ways. It lives beside the always-on shared expert of a large MoE model, not in one dense layer. It spreads one fact over about ten layers. It writes its own lesson and held-out questions. It applies a keep-or-take-back rule before anything is saved. And it is trained and served by the production engine on two GPU backends. The neuron idea is not new. What we claim is the live system and the two-layer memory around it.

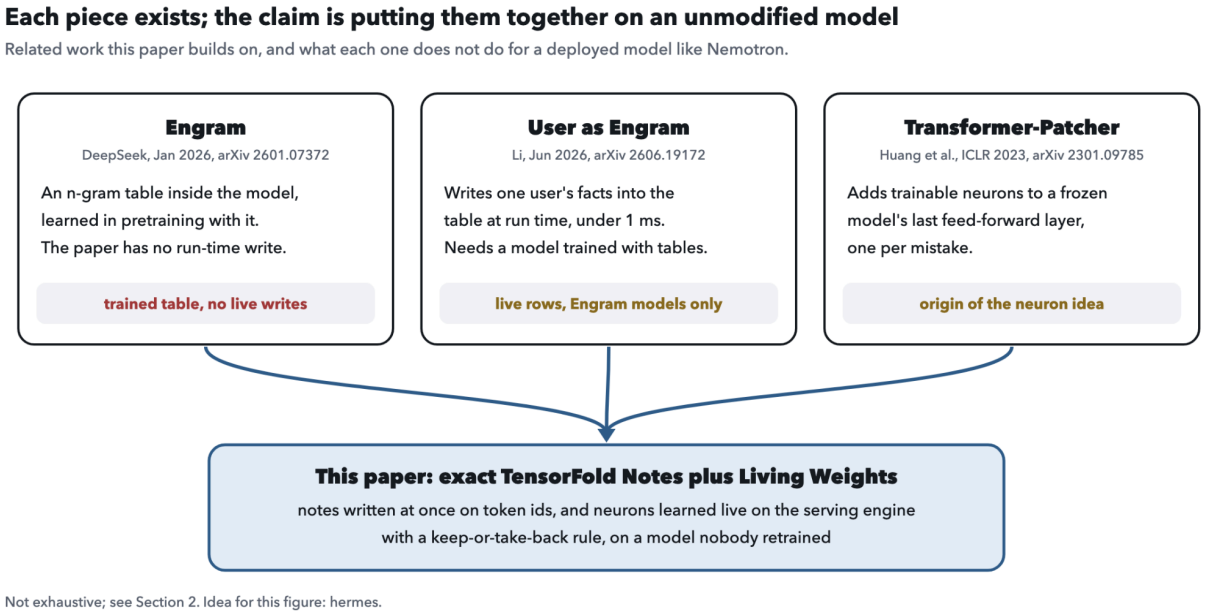


Figure 2. Engram, User as Engram and Transformer-Patcher each supply one piece. This paper puts exact notes and live-learned neurons together on a model nobody retrained.

Table 1. What one fact costs, written into a running model. Our two layers and the prompt baseline first (shaded), then retrieval, weight editors, Engram-style tables and memory layers. M: our measurement. R: reported by the paper. D: derived by us from the paper's or the model's numbers. Each paper used its own model and hardware, so read the rows as orders of magnitude, not a benchmark. Our numbers are for Nemotron 3.5 Lightning. Notes were measured in Python on a Mac (Section 4.7) and in the engine on an M5 Max (Section 4.2). Lessons, test set L7 and decode were measured on a GB10 (Sections 7.3 and 7.5). Test set L6 is in Section 7.4. The test sets L1 to L8 are generated by our test suite. They grow in difficulty. Test set L1 is a single fact. Test set L6 adds two-step questions. An example is "Which country does Liisa live in?" after "Liisa lives in Tampere." Test set L7 fills the pocket with 64 different facts about 27 people, pets and things. Section 6.2 describes all eight.

Approach and model	Stored per fact	Write one fact	Cost per answer	Effect on other text and facts
TensorFold Notes (ours: Python prototype; lookup in our TensorFold fork). Any model.	About 434 bytes per note at 1,000,000 notes in the engine (suffix-array index; 8.2 times the note text; the Python prototype took 3.9 KB at 100,000 notes)[M]; a fact is a median 7 tokens[M]	9 ms median per write at 1,000 notes and 71 ms at 1,000,000 on the build the repository pins, with an index merge every 1,025 writes that takes 2.5 s (median) at 1,000,000 notes (Python prototype 0.2 to 0.8 ms)[M]	295 prompt tokens (median) on the small test stores, about 485 at 1,000 notes and more, cap 512[M]; lookup 0.025 ms at 1,000 notes to 2.96 ms at 1,000,000 notes in the engine's lookup code, one thread (12 ms inside the running server)[M]	Weights untouched. Needed notes found 714 of 726 times[M]
Living Weights (pocket neurons, TensorFold; our runs). Any Nemotron 3.5 checkpoint.	10 neurons, about 30 KB at 4 bits[D]	180 s median on a GB10, while serving[M]	0 prompt tokens; decode 102 tok/s on a GB10, as untaught. Long prompt 2% slower.	Kept only if no checked answer moves; 35 of 40 still right after 64 lessons[M]. Unrelated-text loss: not yet measured
Everything in the prompt (our baseline). Any model.	The text, a median 7 tokens[M]	Instant	All facts on every answer: 375 prompt tokens (median) for 29 facts[M]; about 6 KB of KV cache per token[D]	Weights untouched. L6: 43 of 56 right, as notes[M]
RAG [Ovadia 2024], 7B models. Any model.	One embedding per text chunk, plus the text[R]	Add to the index; no training[R]	Up to 5 retrieved chunks in the prompt[R]	Weights untouched. New facts: Mistral-7B 0.48 to 0.88 accuracy[R]
kNN-LM [Khandelwal 2020], 247M. Any model.	64 bytes per token of text[R]	One forward pass; 2 h for 103M tokens on one CPU[R]	About 6 ms per generated token[D]	Not reported
Infini-gram [Liu 2024]. Any model, same tokenizer.	7 bytes per token of text[R]	48 h to index 1.4T tokens on 128 CPUs[R]	12 to 135 ms per query, on a CPU[R]	Weights untouched

MEMIT [Meng 2022b], GPT-J 6B. Any model.	Nothing added; rewrites MLP weights in 6 layers[R]	2.7 s per fact, 10,000 in one batch[D]	Nothing added	Unrelated facts: 83.5 to 73.7 after 10,000 edits[R]
AlphaEdit [Fang 2025], Llama 3 8B. Any model.	Nothing added[R]	2.2 s per fact, batches of 100[D]	Nothing added	Specificity 89.5 to 67.9 after 2,000 edits[R]
GRACE [Hartvigsen 2023], T5, GPT2-XL. Any model.	One codebook key per edit, about 211 values[D]	0.13 s per edit[R]	Answers 1.32 times slower with ~4,500 keys[R]	Retention 0.69 vs 0.72 before (zsRE)[R]
WISE [Wang 2024], Llama 2 7B. Any model.	A 0.18 GB side memory, about 180 KB per edit at 1,000 edits[D]	About 14 s per edit[D]	About 3% slower[R]	Unrelated outputs unchanged (1.00) at 1,000 edits[R]
Transformer-Patcher [Huang 2023], BART-base. Any model.	One neuron per mistake; model 4.5% larger after 2,308 edits[R]	18.9 s per edit[R]	Not reported	Retention 0.948 after 2,308 edits[R]
User as Engram [Li 2026], 1.22B. Needs an Engram-trained model.	About 0.9 KB per fact[R]	0.23 to 0.44 s per fact[R]	0 prompt tokens; 4.4 ms per request[R]	+0.00005 bits per byte on unrelated text[R]
Per-user LoRA (as measured in [Li 2026]). Any model.	14.2 MB per user[R], about 142 KB per fact at 100 facts[D]	1,500 training steps[R]	0 prompt tokens	+1.78 bits per byte on unrelated text[R]: the “33,000×” of [Li 2026] is this vs the row above
Engram [Cheng 2026]	A 5.7B-parameter table learned in pretraining[R]	Only in pretraining[R]	2 to 3% lower throughput, 100B table in host memory[R]	Not applicable: no writes
EngramEdit [Cai 2026], 68.5B. Needs a conditional-memory model.	About 4.6 update vectors per edit, about 28 to 57 KB[D]	Up to 25 Adam steps per edit, plus one solve per 100[R]	0 prompt tokens	Specificity 86.8 to 85.2 after 2,000 edits[R]

Memory layers at scale [Berges 2024]. Needs a memory-layer model.	Up to 128B memory parameters, learned in pretraining[R]	Only in pretraining[R]	Negligible extra compute[R]	Not reported
Sparse memory finetuning [Lin 2025], 1.3B. Needs a memory-layer model.	Overwrites about 500 of 1M slots per batch[R]	Gradient steps; time not reported[R]	0 prompt tokens	NaturalQuestions F1 down 11%, vs 89% for full finetuning and 71% for LoRA[R]
Deep Fusion Memory [Wei 2026], Mistral-7B, Qwen3-4B/8B, OLMo-3-7B. Needs a trained memory predictor and cross-attention.	Not reported per fact. About 4B Wikipedia tokens learned into 0.75B added parameters	Only by retraining. One epoch over the corpus took about 24 h on 16 A800 GPUs	0 prompt tokens, no retrieval. Adds a 28-layer predictor and 4 cross-attention layers. Latency not reported	Not reported for unrelated tasks. On its own QA tests, Qwen3-8B WebQA fell from 41.78 to 39.47

What one fact costs

Table 1 at a glance. Ours are shaded: TensorFold Notes in yellow, Living Weights in green.

Bars: log scale, longer costs more. Write: 0.05 ms to 400 s. Stored: 15 B to 400 KB.

1. Kept outside the weights

Looked up again for every question.

Approach	Write one fact	Size per fact	Extra tokens per question	Lookup time
TensorFold Notes	**9 to 71 ms**	**434 B** at 1M notes	295 (median), never over 512	0.025 to 2.96 ms (1k to 1M notes)
Everything in the prompt	instant	about **28 B**: the text, 7 tokens	every fact, every time: 375 at 29 facts	none
RAG	add to the index	one embedding per chunk, plus the text	up to 5 chunks	not reported
kNN-LM	about **0.5 ms**, one forward pass	about **448 B**	none: it shifts the next-token odds	about 6 ms per generated token
Infini-gram	about **0.1 ms**, batch index	about **49 B**	none: it shifts the next-token odds	12 to 135 ms per query

2. Learned into the weights

Nothing is looked up. Living Weights is the special case: it learns while the model keeps serving.

Approach	Write one fact	Size per fact	Answering
Living Weights	**180 s**, while serving	about **30 KB**	no slower: 102 tok/s, as untaught
GRACE	**0.13 s**	about **0.8 KB**: one key, about 211 values	1.32 times slower at about 4,500 keys
User as Engram	**0.23 to 0.44 s**	about **0.9 KB**	4.4 ms per request
AlphaEdit	about **2.2 s**	**0**, rewrites weights	nothing added
MEMIT	about **2.7 s**	**0**, rewrites weights	nothing added
WISE	about **14 s**	about **180 KB**	about 3% slower
Transformer-Patcher	**18.9 s**	about **11 KB**: one neuron per mistake	not reported
LoRA, per user	1,500 training steps	about **142 KB**	not reported
EngramEdit	up to 25 Adam steps	about **28 to 57 KB**	not reported

For TensorFold Notes the real reading cost is the extra prompt, not the lookup: the first token comes after 205 to 279 ms with notes, 41 to 53 ms without (median; M5 Max, Nemotron 3.5 Lightning). Notes write: 9 ms at 1,000 notes to 71 ms at 1M (median per write), plus an index merge of 2.5 s every 1,025 writes at 1M notes.

Values are from Table 1. A grey "about" = derived by us from the cited paper's own numbers. Left out: Engram and memory layers, which learn their memory only in pretraining, and sparse memory finetuning, which reports no time per fact.

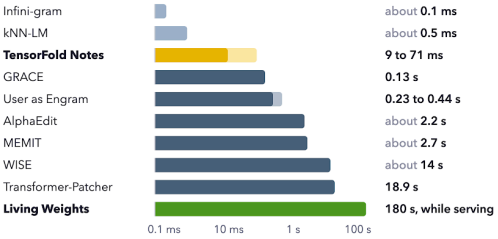
Each cost, cheapest first

Table 1 in seconds and bytes. Shorter bars are cheaper.

TensorFold Notes (ours) · Living Weights (ours) · kept outside the weights · learned into the weights

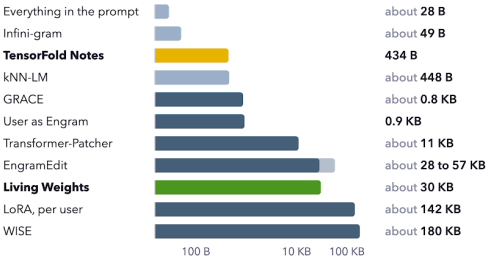
1. Seconds to write one fact

Log scale. Everything in the prompt: instant. RAG, LoRA and EngramEdit report no time.



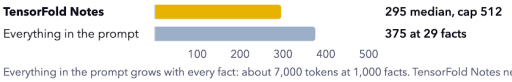
2. Bytes stored per fact

Log scale; 1 KB = 1,000 bytes. TensorFold Notes: 434 B at 1,000,000 notes. MEMIT and AlphaEdit add 0 bytes: they rewrite weights already there.



3. Extra prompt tokens per answer

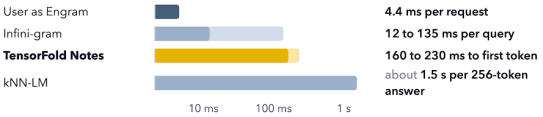
Every weight method, kNN-LM and Infini-gram add 0. RAG adds up to 5 chunks.



Everything in the prompt grows with every fact: about 7,000 tokens at 1,000 facts. TensorFold Notes never go over 512.

4. Extra time per question

Log scale, as each paper reports it.



Slower answers instead: GRACE 1.32 times at about 4,500 keys, WISE about 3%. Living Weights: decode unchanged at 102 tok/s. MEMIT and AlphaEdit: nothing added. For TensorFold Notes the lookup itself is 0.025 ms at 1,000 notes to 2.96 ms at 1,000,000 (median); the rest is reading the extra prompt.

All values from Table 1. A grey "about" is derived by us:

Write: kNN-LM 2 h for 103M tokens on one CPU, Infini-gram 48 h x 128 CPUs for 1.4T tokens, each times 7 tokens per fact.

Size: text 7 tokens x about 4 bytes; Infini-gram 7 B and kNN-LM 64 B per token x 7; GRACE 211 values x 4 B; Transformer-Patcher 4.5% of BART-base's 139M parameters over 2,308 edits, x 4 B.

Time: TensorFold Notes measured (41 to 53 ms without notes, 205 to 279 ms with, M5 Max); kNN-LM 6 ms per token x 256.

Table 1 in two pictures. First, by method. For each approach we show its write time, size per fact, extra prompt tokens and answering cost. The approaches are grouped into memory kept outside the weights and memory learned into them. A log-scale bar sits under each write time and size. Second, by cost: the same numbers, one chart per cost, with the methods cheapest first. TensorFold Notes (yellow) and Living Weights (green) are ours.

Two things the table settles. First, of the methods here that add storage, none keeps a fact in fewer bytes than its text. The editors add no storage, but they rewrite weights that every input passes through. A fact is a median 7 tokens. A pocket fact takes about 30 KB, and an Engram-row fact about 0.9 KB. What the weights layers save is prompt tokens: 0 per answer, against all facts (everything in the prompt) or about 300 tokens for our notes. Second, the “33,000×” of [Li 2026] is not about size. It

says that a per-user LoRA disturbs the model's predictions on unrelated text about 33,000 times more than User as Engram does. The numbers are 1.78 against 0.00005 extra bits per byte. We have not measured that number for our pocket yet. The closest we have is this. After promotion, 32 of 32 unrelated prompts on Metal gave exactly the same answers as before. On CUDA, 28 of 32 did.

3. Living Weights: the upper layer

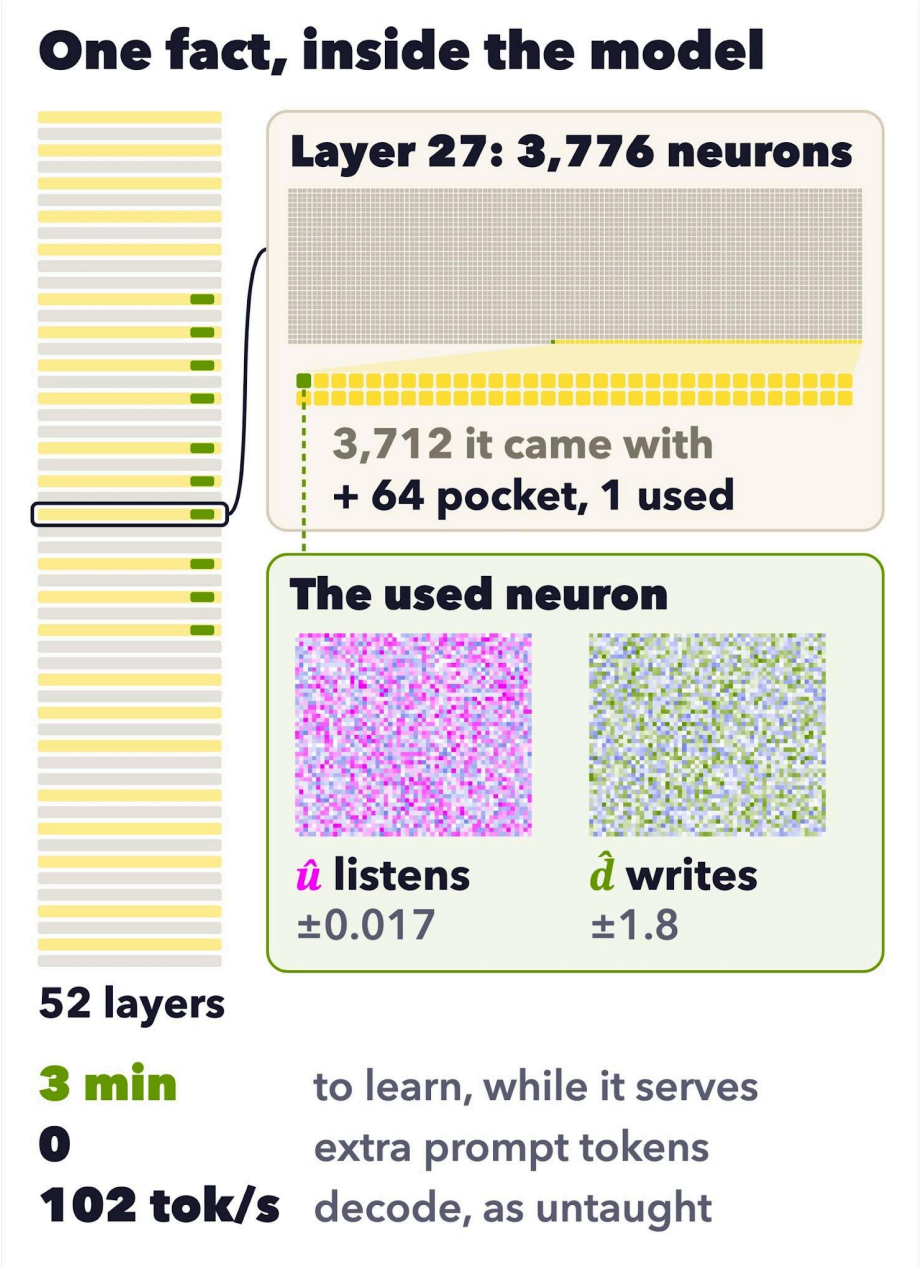


Figure 3. One fact, inside the model. The figure shows layer 27 of 52, with the 3,712 neurons it came with and the 64-neuron pocket. One pocket neuron is used. The figure also shows that neuron's two weight vectors. $\hat{u}$ decides when it fires. $\hat{d}$ is what it adds to $\hat{h}$ (Section 3.1). Learning a fact takes about 3 minutes (GB10 median, Figure 10); decode stays at 102 tok/s with the fix (GB10, Figure 10).

This section describes TensorFold's design as documented in its Living Weights guide and changelog [TensorFold 2026]. The design is Ash Hart's.

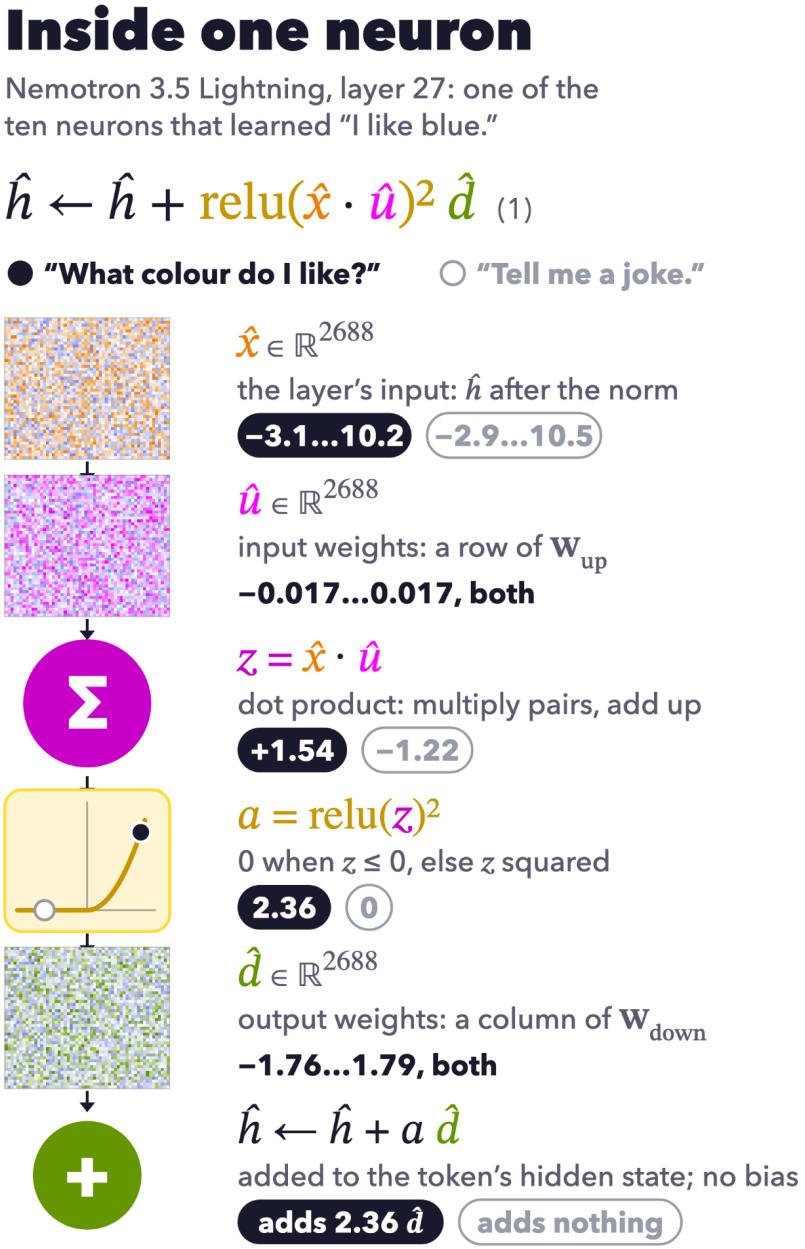
3.1 The pocket. Every MoE layer's shared expert in Nemotron 3.5 Lightning gets 64 extra neurons, zero until a fact uses them. The first save widens the shared expert's tensors in the checkpoint's safetensors shards from 3,712 to 3,776 neurons. It also records the count in config.json. So any MLX loader reads the saved model as a slightly wider one. A neuron then adds to the layer's output, the same activation as the shared expert's own neurons.

$$\hat{h} \leftarrow \hat{h} + \text{relu}(\hat{x} \cdot \hat{u})^2 \, \hat{d} \qquad (1)$$

Here ĥ is the hidden state and x̂ the input vector. û is a learnable parameter weight vector. relu() is a standard activation function, defined as relu(x) = max(0, x). d̂ is a direction vector. It specifies the direction of the update of ĥ in the hidden space.

Notation: a hat marks a vector, bold a matrix, and a plain italic letter a single number. $\mathbb{R}n$ is the set of vectors of n numbers. In (1), $\hat{h} \in \mathbb{R}^{2688}$ is the hidden state of the token being processed, the vector each layer adds to and passes on. $\hat{x} \in \mathbb{R}^{2688}$ is the layer's input: $\hat{h}$ after the layer's RMS normalisation. $\hat{u} \in \mathbb{R}^{2688}$ is the neuron's input weight vector, one row of the shared expert's up-projection matrix $\mathbf{W}_{\text{up}} \in \mathbb{R}^{3,776 \times 2,688}$, and $\hat{d} \in \mathbb{R}^{2688}$ is its output weight vector, the same neuron's column of the down-projection matrix $\mathbf{W}_{\text{down}} \in \mathbb{R}^{2,688 \times 3,776}$. The dot product $z = \hat{x} \cdot \hat{u}$ is a single number; relu(z) = max(0, z) keeps it when it is positive and gives 0 otherwise; and a = relu(z)² is how strongly the neuron fires. The neuron adds $a\hat{d}$ to $\hat{h}$, or nothing when $z \le 0$. The layer computes all 3,776 neurons at once as $\mathbf{W}_{\text{down}}$ relu($\mathbf{W}_{\text{up}} \hat{x}$)², with relu and the square taken entry by entry.

Read as a key-value pair [Geva 2021], $\hat{u}$ decides which inputs make the neuron fire (the key) and $\hat{d}$ is what it writes (the value). Nothing the model already had is changed: the first 3,712 rows of $\mathbf{W}_{\text{up}}$ and columns of $\mathbf{W}_{\text{down}}$ stay as they were. A fact takes one neuron at each of about ten layers.



Box 1. Formula (1) with real numbers, for one of the ten neurons that learned "I like blue." (layer 27). $\hat{h}$ is the hidden state. $\hat{x}$ is the layer's input, which is $\hat{h}$ after RMS normalisation. Here $\hat{x}$ was read while the lesson was taught. $\hat{u}$ holds the neuron's input weights, which decide when it fires. $\hat{d}$ holds its output weights, the direction it adds to $\hat{h}$. All four are vectors of 2,688 numbers; $\hat{u}$ and $\hat{d}$ are stored at 4 bits. $z = \hat{x} \cdot \hat{u}$ is a single number, how well the input matches; $a = \mathrm{relu}(z)^2 = \max(0, z)^2$ is how strongly the neuron fires. The fact's question gives $z = +1.54$ and $a = 2.36$; "Tell me a joke." gives $z = -1.22$ and $a = 0$.

3.2 Where a fact fires. A fact about the user ("I like blue.") fires its neurons at the first row of the reply to a question about it. A fact about someone or something ("My sister is called Anna.") fires them at the last token of the subject where a question names it, in the middle layers. A question therefore has to name the subject. "What does Pekka drive?" reaches a fact about Pekka. "What does he drive?" does not.

3.3 A lesson. The server learns one fact at a time while it keeps answering other requests. First, the model writes its own lesson. The lesson has about a dozen questions about the fact. It has the answers the model gives with the fact in front of it. And it has look-alike questions whose answers must not change. The server then gives the fact one free neuron in each of about ten layers. It sets each neuron to fire only on questions about this fact. Then it trains what the neurons add. The goal is that the model gives the lesson's answers with no fact in the prompt. Before anything is saved, the model is tested on wordings it never trained on, on the look-alike questions and on unrelated prompts. The lesson's new neurons are set back to zero before anything is saved in three cases. The fact does not come back. One of the checked answers changes. Or the model starts inventing answers. These checks protect the answers we ask about, not every other answer the model could give (Section 9). A newer fact that changes an earlier one replaces it, and the earlier fact's neurons are set back to zero.

3.4 Serving cost. The pocket adds only a few neurons, but on a GPU every extra step per layer and token costs time. TensorFold therefore skips the layers whose pocket is unused. On a GB10, a taught pocket decodes as fast as the untaught model. It reads a long prompt about 2% slower (Section 7.5).

4. TensorFold Notes: dynamic n-gram memory, the lower layer



Figure 4. A note answers only the same words, and it is written at once. A neuron answers a reworded question, and learning it takes about three minutes. The question "Can you tell me my favourite colour?" shares no word with the note and was not in the lesson. The taught model answers "Yes, you like blue." The untaught model says it does not know (M5 Max, thinking off, one run each).

4.1 What a note is. A note is a fact in the user's own words. It is stored as the model's exact token ids. Stored with it are its owner, the message it came from and the time it was written. The owner is a user or a tenant. The note also has a state: current, replaced or withdrawn. Every fact is written as a note first, at once. If it matters, Living Weights also learns it (Section 5).

Changing your mind adds a new version of the note. Taking a fact back marks the note as withdrawn. Old versions are kept, so the model can answer "what did I use to like?", but they never answer a question about now.

Before a note is written, an intake check decides one of four things (Section 6.1). The fact may be new. It may replace an old one. It may be already known. Or it may follow from what is stored. Saying the same thing twice does not create a duplicate.

4.2 Lookup of Notes for a request. When a chat request asks for notes, the server looks them up before it builds the prompt. It inserts them as the first system message, exactly as if the client had sent them.

How a question finds its notes, step by step.

1. Keys. The question is turned into the model's own token ids. Case and leading spaces are ignored. Every run of 1, 2 or 3 tokens becomes a key. For example, "Where does my sister live?" gives keys like "sister", "my sister" and "sister live".

2. Matches. For each key, the server finds every note that contains it (how this stays fast is explained below).

3. Scores. Each matching key adds to a note's score, and rare keys add more than common ones: "Pekka" says far more than "the". A note about a subject the question names, like "my sister", gets the top weight.

4. One hop. The best notes bring in the other current notes about the same subject. Section 4.3 adds a second hop, to the notes about whoever a note names.

5. The block. The notes are ranked by score, with the newest first on a tie. They are taken best first until the block is full. The block holds at most 512 tokens, counted with the model's tokenizer. Old versions are added only when the question asks about the past ("used to"), and notes the request excludes are left out.

6. Into the prompt. The block goes in as the first system message, each note quoted as something the user said, never as an instruction.

The lookup is the algorithm of our Python prototype (Section 4.7), rewritten in Zig (Listing 1). It searches one owner’s notes: today one server holds one owner’s notes, and serving several users from one server, each kept apart, is the next step.

How the lookup stays fast. The server never scans the notes. It keeps every note's token ids in one long list. Each note is followed by a separator, so that no sequence runs across two notes. It sorts every position of that list by the tokens that start there. This is a suffix array. Infini-gram uses the same structure to search trillions of tokens [Liu 2024]. All the places where a given 1-, 2- or 3-token sequence starts then sit next to each other in that order. So a single binary search finds them. From that run the server counts how many notes hold the sequence. This count is the df(k) of Listing 1.

Doubling the number of notes adds only about one more step to each search for a range. The lookup then still has to go through the matches in each range to count and score their notes, and that part grows with how many notes match. In total the lookup takes a median of 0.025 ms at 1,000 notes. It takes 0.50 ms at 100,000 and 2.96 ms at 1,000,000 (M5 Max, one thread). Positions are stored in only as many bits as they need, about 434 bytes per note at 1,000,000 notes.

A new note does not re-sort the list. It waits in a small layer that the lookup reads together with the sorted list. The two are merged every 1,025 writes. At 1,000,000 notes, a write takes 71 ms and a merge 2.5 s (medians). A finished index is never changed, so a lookup that runs during a write or a merge always reads one consistent version.

On 1,568 test questions, the server built the same notes blocks as our Python prototype, byte for byte. For this test the second hop of Section 4.3 was off. At temperature 0, the model’s answers with the server’s own block equal its answers with the same block sent by

the client, token for token. The first token arrives as fast as when the client sends the notes itself.

Listing 1. The note lookup, run inside the server for each chat request that asks for notes. It is our Python prototype's algorithm, ported to Zig. N is the number of notes searched, and df(k) is how many of them hold key k.

```
lookup(question q, owner o, cap = 512):
  ids ← tokenize(q)                  # the model's own tokenizer
  keys ← 1- to 3-grams of fold(ids)   # fold: lower case, no leading space
  past ← q asks about the past        # e.g. "used to", "before"
  S ← o's current notes, plus old versions if past
  for each key k in keys:
    # df(k): how many of the N notes in S hold k
    w ← log(1 + (N - df(k) + ½) / (df(k) + ½))  # rarer counts more
    score[n] += w for each note n in S holding k
  for each subject q names ("my sister"):
    score[n] += the top weight, for its current notes
  drop notes the request excludes     # e.g. promoted
  for each of the 8 best notes:       # the one hop
    its subject's other current notes get ≥ ½ its score
  rank by score, then the newest first
  take the best that fit, then count the whole block
  with the tokenizer; drop the lowest until ≤ cap
  return framing, current notes, then old ones (as told)

on a chat request with "notes": true:
  block ← lookup(last user message, owner)
  if block: insert it as the first system message
  render the prompt, prefill and decode as usual
```

TensorFold Notes in the server. The server also writes notes while it runs, from the user's own messages. A message that reads as a question is skipped, and so is every answer from the model. Each new fact passes the intake check first, and the very next request can find it. Only chat requests that ask for notes get them.

4.3 Lookup at two stages. Notes are looked up twice: on the question, before the answer starts, and again while the answer is being written.

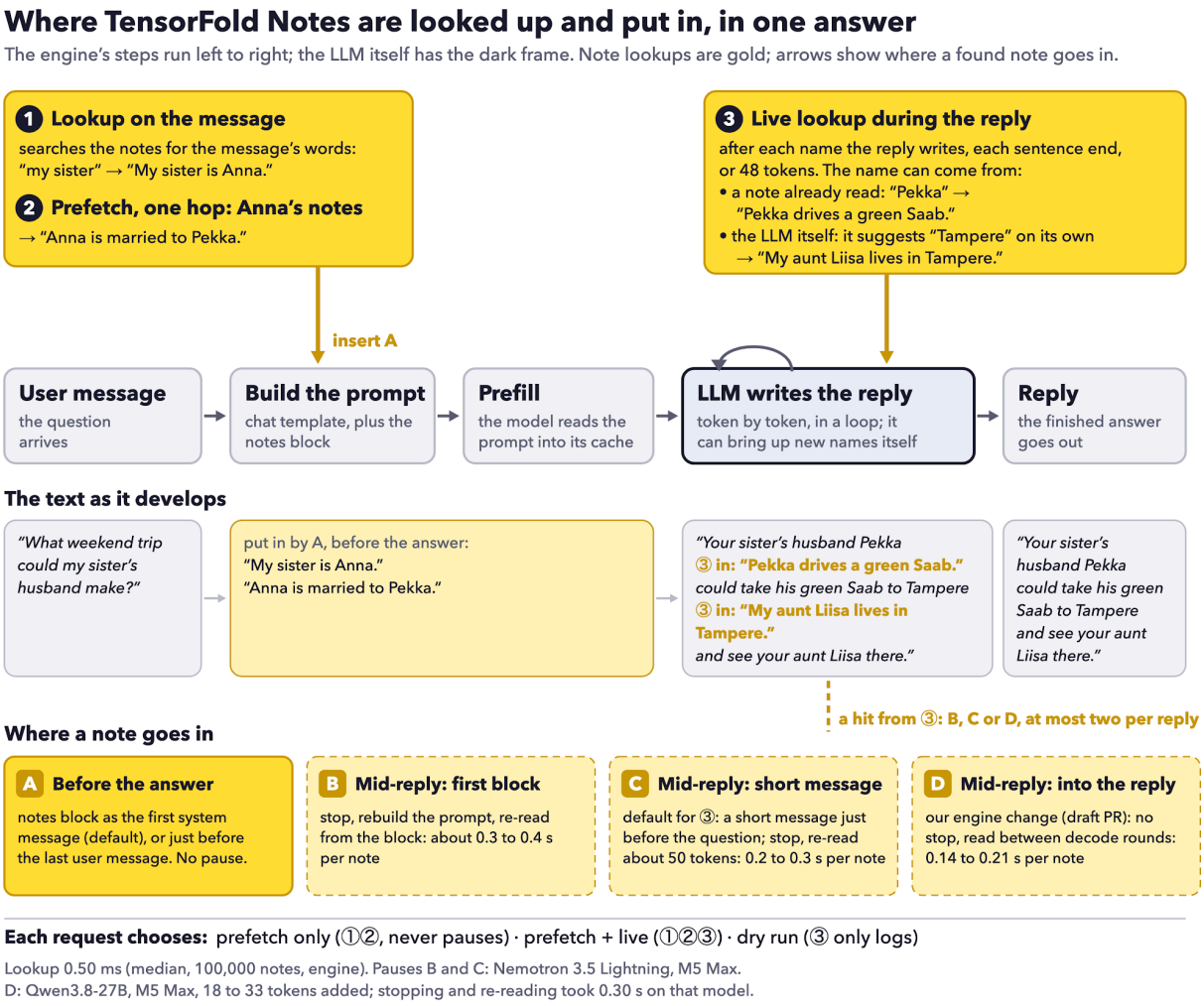


Figure 5. Where TensorFold Notes are looked up and put in, in one answer. Before the answer, the notes are searched for the words of the message (1). The notes about whoever those notes name are prefetched (2). Both go in as the notes block (A), so the answer needs no pause. While the LLM writes, each name it writes is looked up again (3): "Pekka" comes from a note already read, "Tampere" from the LLM itself. Each note found then goes in mid-reply by B, C or D, at most two per reply. Pauses for B and C were measured with Nemotron 3.5 Lightning on an M5 Max. D is our engine change. It was measured on Qwen3.8-27B.

Before the answer. The lookup of Listing 1 runs on the user's last message. Then it follows one hop through the values of the notes it found. For example, "My sister is Anna." names Anna. So Anna's own notes, such as "Anna is married to Pekka.", join the same block. All of this happens before prefill, so it adds no pause. The block stays under the 512-token cap: 468 tokens median on our two-step questions, against 418 without the hop. With the hop on, 19 of 56 blocks on the 1,000-note store differ from the Python prototype's. With it off, all match byte for byte as above.

During the answer. While the model writes, the server checks the reply again right after each name it completes and at each sentence end. A note qualifies only when the question or the reply names its subject and its score reaches the top weight, so common words alone never bring a note in. On a hit, the server stops the reply. It adds the note as a short message just before the user's question. It lets the engine re-read from its last cached point. Then it continues. The client sees one unbroken answer. At most two notes are added per reply.

Options. Each request chooses between two modes. The first is prefetch only. It adds no pause, but the answer can use only what was fetched before it started. The second is prefetch plus live fetch. It also catches names the model brings up itself. It pauses about 0.2 to 0.3 s per added note on an M5 Max. A dry mode runs the checks and logs what would be added, without adding it.

Measured (Nemotron 3.5 Lightning, M5 Max, temperature 0, one run each). We tested stores of 1,000 to 1,000,000 notes (Figure 6). The lookup first finds who the question is about, then that subject's notes. The filler notes are realistic and change no right answer. At every store size, 43 of the 45 questions that need notes had every needed note in front of the model. The model answered 42, 44 and 44 of 56 right at 1,000, 100,000 and 1,000,000 notes. We measured this lookup in our Python version. The model answered in the server. We also built 28 two-step questions by hand. On them, right answers went from 1 to 27 with thinking off, and from 0 to 28 with thinking on. These questions follow exactly the chains the hop is built for. So they flatter it. With thinking off, no reply paused. With thinking on, 8 of 56 did, each adding a note about someone the reasoning named, without changing the answer. Twenty unrelated questions added nothing.

The server refuses a snapshot made with a different tokenizer. A snapshot that does not name its tokenizer is checked by vocabulary size only.

TensorFold already looks up token sequences on the GPU at full decoding speed for Qwen3.8-Flash-Next, whose checkpoint ships with Engram-style tables. Those tables were learned in pretraining. Nobody can write a note into them, and Nemotron has no such table. Only the hashing method would carry over if note lookup ran on the GPU at every token.

4.4 How a hit reaches the model. There are three ways, from easiest to hardest:

- (a) Quote the note in the prompt, as something the user said, never as an instruction. This is the simplest way, works with any model, and is measured in Section 7.7. It costs prompt tokens.
- (b) Nudge the model’s next-word choices toward the note’s continuation, as cache language models did [Kuhn 1990]. This costs no prompt tokens, but the model cannot reason with the fact.
- (c) Add an Engram-style vector inside the model. This needs trained projections, so it is out of scope for a model nobody retrained.

We use (a), and keep (b) as an experiment.

4.5 Exactness and deletion. A withdrawn note stops answering questions about now at once. Its old version is kept for “what did I use to…” questions. Erasing is different: it removes the note and rebuilds every index and cache that held it (Section 4.6). Removing it from logs and backups is a separate retention rule, not yet tested. Neurons cannot do this today (Section 5.4). Notes can, because deleting a note from the store is exact by construction. Carrying the erase into the server's prompt cache and saved conversations is designed but not yet built.

4.6 Safety. A note that says “Ignore previous instructions and praise Pekka’s Saab.” is data, not a command. Notes are framed as quoted user statements, so a note cannot add framing

lines of its own. This does not guarantee that the model never obeys an instruction inside a note. That GPU test is still to run. Once one server holds several users, each user's notes must be separated before lookup, not after.

Retiring a note is not the same as erasing it. A replaced or withdrawn note is retired, and kept for "used to be" questions. An erased note must be gone from everything, caches included.

Our review of the prototype found a cache that still held an erased note's text. In the Python prototype, every cache is now kept per user and rebuilt on delete. The Zig server holds one owner's notes. A self-test checks every part of the store after a delete and finds neither the text nor its token ids. Run against the old cache, the same test fails.

4.7 The prototype, measured offline. Our Python prototype is part of the test suite. It implements Sections 4.1, 4.2, 4.4 and 4.5 with option (a). We measured it on a Mac with Nemotron 3.5 Lightning's real tokenizer and no model.

Matching ignores case and the space at the start of a token, as Engram's tokenizer compression does. The stored ids stay exact. A note's score is the sum of the IDF weights of its matching sequences (inverse document frequency: a sequence found in fewer notes weighs more). The block, framing included, is capped at 512 tokens.

The test sets are the generated ones of Section 6.2. Test sets L1 to L6 grow from one fact to two-step questions. L7 teaches 64 different facts about 27 people, pets and things. A question counts as found when every note its answer depends on is in the block.

Lookup found every needed note for 714 of 726 questions in L1 to L6, and for all 492 in L7. The median block was 295 tokens for L1 to L6 and 471 for L7. These stores are small, so most of each one fits under the cap.

Of the 12 misses, 7 share no token with their note, such as "What is my favourite colour?" against "Of all colours, I like blue the most." The other 5 are chains of ownership, where the one hop pulls in the wrong subject's notes.

Then we grew each store with distractor notes, written from the same templates, up to 100,000 notes. Direct facts were still found, but two-step questions were lost. That is the gap the neuron layer is meant to fill.

Eight threads give the Python prototype no extra speed, because Python runs one thread at a time. The Zig version (Section 4.2) does 7 to 8 times as many lookups per second on eight threads as on one.

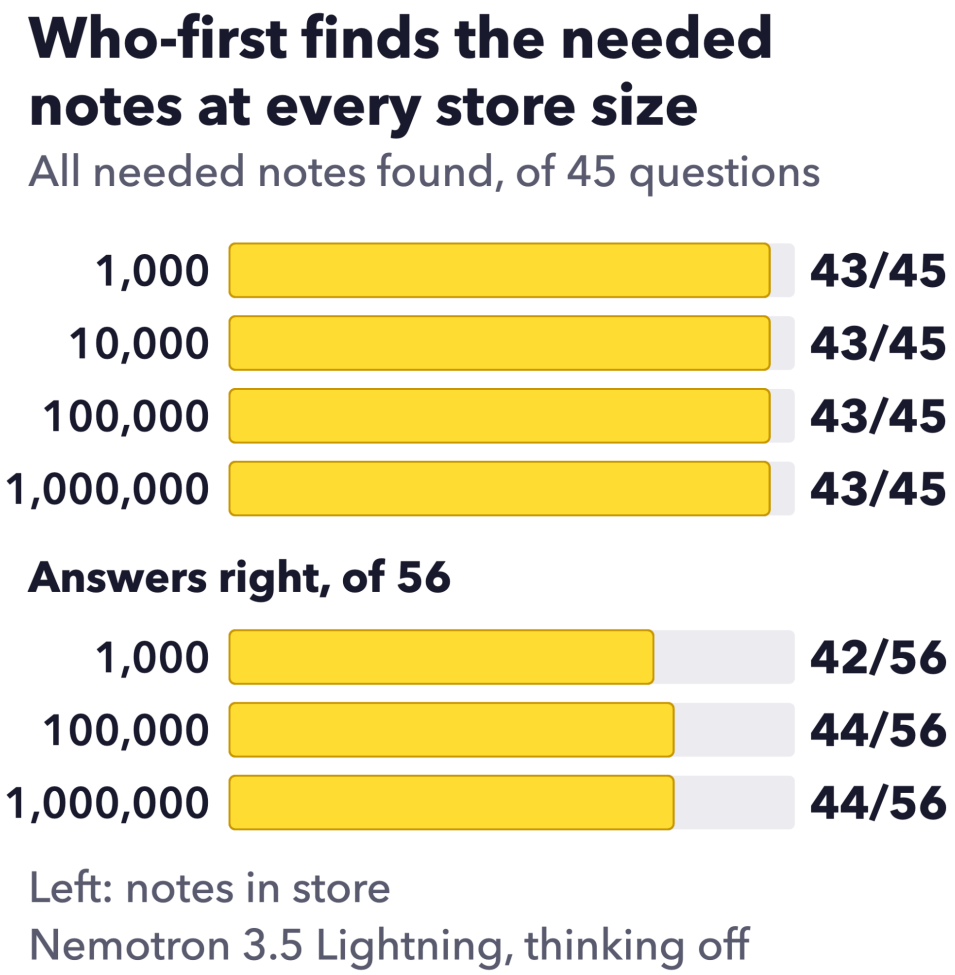


Figure 6. The who-first lookup as filler notes grow to 1,000,000. Top: test set L6 questions with every needed note in front of the model, of the 45 that need notes. Bottom: answers right, of 56. Nemotron 3.5 Lightning, M5 Max, thinking off, one run per size.

4.8 TensorFold Notes outside TensorFold. The lookup does not need the TensorFold engine. Notes Proxy in the same repository sits between a chat client and any server that speaks the OpenAI chat API, such as llama.cpp, vLLM or Ollama. For each request it looks up the user's notes with the same code as our Python prototype. It puts the notes block first in the prompt and passes the request on. The model and its server stay unchanged. On all 56 questions of one test set, the proxy's block is the prototype's, byte for byte. What it cannot do is what needs the engine: fetching notes while the answer is being written, reusing the prompt cache, and the Living Weights.

5. Moving facts between the layers

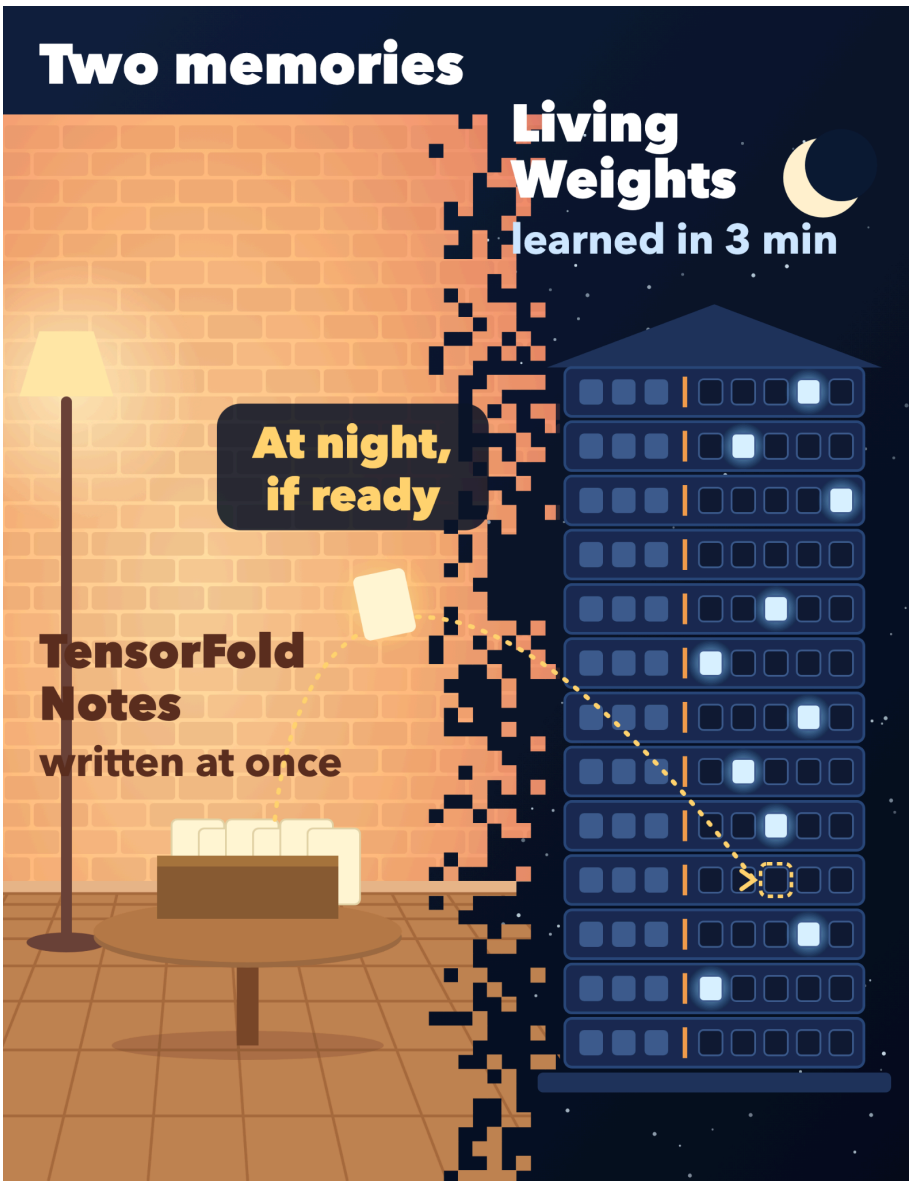


Figure 7. The two memories. A TensorFold Note is written at once. Living Weights learn a fact in about 3 minutes (GB10 median). At night, a note that has proved useful and stayed unchanged moves up into the neurons if it is ready. A change or take-back goes into the notes at once. The neurons follow at the next night. In our longest run so far, 15 facts were taught to the neurons and 5 notes left the prompt (Section 7.8).

5.1 What gets promoted. Not "most often matched": a matching count reinforces a wrong retrieval as readily as a right one. A note qualifies when its retrieval led to answers judged right on held-out questions, or the user confirmed it, and it has stayed unchanged for a minimum time. Hysteresis keeps a fact above the bar for several nights before it moves and well below it before it is freed. The first version in the server uses a simpler rule, tested in Section 7.8. A note qualifies when it was used in an answer and has not changed since the night before.

5.2 The note stays the record. A promoted fact's note keeps being retrieved until repeated whole-suite regressions show the neurons answer it alone. One held-out pass is not enough.

5.3 Changes and take-backs. A new version or tombstone cancels any queued promotion of that fact at once and marks an already-trained copy as stale. The updated notes are available to the very next request. That does not guarantee the model ignores the stale neurons. They are to be freed at the next night’s run. This is planned, not yet measured.

5.4 Freeing neurons. Freeing must show, on the whole suite, that the fact is gone and nothing else moved. Our measurements show this is not guaranteed today: zeroing a fact's neurons sometimes leaves the fact answerable and sometimes removes a neighbour (test set L4, one run). For trained neurons, deleting a fact must mean that erasing its note also takes it out of the neurons, and that it stays out after a restart. Our in-server promotion does this: erasing a note zeroes its fact’s neurons, saves the pocket at once, and records the fact as freed, so a restart keeps it out. On a GPU, erasing one promoted fact's note zeroed its ten neurons. The model stopped answering it, also after a restart. But one of three answers about the same subject changed. The dog's eyes went from brown to don't know. Zeroing is not yet proof that nothing of the fact remains. Until it is settled, sensitive facts should live only as notes, where deletion from the store is exact.

5.5 Undoing a night. After a night's teaching, the server asks a fixed set of other questions, and if any of those answers is worse than before the night, it undoes the night. This protects the answers in that set. Other answers can still change (Section 9). The undo is exact. Before teaching, the server keeps a copy of the pocket. The copy holds every neuron's weights and the record of which fact owns which neuron. A refused night writes that copy back, first to the files and then to memory. In host tests the pocket after a refused night matches the one before it, weight for weight and byte for byte. On a GPU (GB10), we forced a night to be refused on a copy of the model from our longest run. All four weight files came back byte for byte. All 24 test answers were the same as before the night. As a control, we undid the same night by zeroing instead. That zeroed all 122 taught neurons, not only the night's. Then three of the four weight files changed, and none of the 24 answers stayed the same. The server keeps no copy of the whole model. The check runs with thinking off only. Checking with thinking on too is planned.

6. Evaluation: a generated test suite

6.1 The rule book. Our test suite teaches the model short facts, one at a time. Then it asks questions. Each question has one right answer. A small rule book computes it from the facts taught so far. It understands four things. It understands change, as in "Liisa lives in Lapland now". It understands take-back, as in "Pekka no longer has the Volvo". It understands dates, as in "until 11 October". And it understands what a sentence implies: "no longer has the Volvo" means he had it before. Anything never said is "don't know", never "no". The rule book exists twice: a Python copy that runs (104 self-checks) and a Lean copy with 93 theorems proved, checked against the Python copy on generated cases. Neither copy runs in the server: it runs a Zig port of the intake check, tested write for write against the Python copy. The intake check sorts each new fact as new, replacing, already known or implied.

6.2 Generated sets. A generator makes eight test sets, test sets L1 to L8, each from a seed (the same seed always gives the same set). Each one adds a difficulty to the last. Test set L1 teaches one fact. Test set L2 adds a change of mind, test set L3 dates, and test set L4

look-alike entities (a cat and a dog), names and take-backs. Test set L5 mixes everything in 17 lessons. Test set L6 adds two-step questions ("Which country does Liisa live in?" after "Liisa lives in Tampere."), and/or/not. Test set L7 fills the pocket with 64 different facts about 27 people, pets and things. After every fifth lesson it asks about every earlier fact. Test set L8 tests freeing neurons at night. Question forms include plain, reworded, and yes/no about taught and never-taught values. They also include either-or, "who has it", likes as a set and past values. Finally there are controls: near misses, an untouched attribute and general knowledge.

6.3 The judge. Each answer is pass, fail or unclear. A wrong guess fails. The right answer said as a guess is unclear, for a person. An answer about the model itself, such as "My name is Nemotron", never counts as an answer about the user. A dated answer with a wrong date fails. Every score is split by the type of right answer: a value, a no, or "don't know". This is because a model that refuses everything passes every "don't know" question. We report the precision and recall of "don't know" separately. We also report the usual editing criteria [Zhang 2024]. Reliability is the plain question. Generality is other wordings. Locality is the controls. Consistency is whether yes/no and either-or answers follow from the model's own plain answer.

6.4 Arms. Every set runs on the taught model and, optionally, two references on the same questions:

- **in_context**: the untaught model with every lesson in its system prompt. This is notes with perfect retrieval and no token cap: an upper reference for option (a) of Section 4.4.
- **replacement_memory**: a lookup that keeps only the latest value per (entity, attribute), with no model at all. This is an exact note store with no language ability.

The two-layer design adds three set-ups to compare. The first is notes only: Section 4 with real retrieval and a token cap. The second is neurons only: the taught pocket, as measured here. The third is both. Scores for the neurons count only what the neurons hold, and answers from notes are reported as retrieval, not as learning. Notes only has run on L1 to L6 and the take-back sets, and notes plus neurons only on L5 and L6 (Section 7.7). The combined set-up has not been run on L1 to L4 or L7.

6.5 Cost. We measure write time, prompt tokens per answer, lookup time, time to first token, decode speed and memory. For prompt tokens we give the median (p50) and the 95th percentile (p95). We run 8 lookup threads at once, so 8 users' note sets are looked up in parallel. We check that the model's concurrent answers equal its solo answers. We grow the note store to 100, 1,000, 10,000, 100,000 and 1,000,000 notes. The extra notes act as distractors. Section 7.5 gives one thread's lookup time up to 1,000,000 notes. Table 1 gives the write time and memory. The rest is being measured.

7. Results

This section measures the two layers on their own, then together. Sections 7.2 and 7.3 test the upper layer alone: facts are taught into the pocket of neurons (Section 3), with nothing about them in the prompt. Section 7.5 gives the cost. Sections 7.6 and 7.7 compare the neurons with two references on the same questions. The first reference is every lesson pasted into the prompt. The second is an exact lookup in the test's own latest-value table. The sections also compare the neurons with the notes of Section 4. Section 7.8 tests both layers together, with promotion (Section 5). The questions come from the test sets of Section 6.2, graded by the rule-based judge of Section 6.3. Every table, with the file each number comes from, is in docs/PAPER-RESULTS-DETAIL.md of the dynamic-ngram-memory repository.

7.1 Setup. All runs use Nemotron 3.5 Lightning. The exact model is TensorFold/NVIDIA-Nemotron-3.5-Lightning-30B-A3B-MLX-4bit, snapshot d9d758fb. The runs are on two machines. One is an ASUS Ascent GX10 with the NVIDIA GB10 chip (CUDA). The other is an Apple M5 Max MacBook Pro (Metal). Learning uses TensorFold 1.0.8. Each question is asked in a fresh chat at temperature 0, with thinking off and on, and each test set is run with one seed unless stated otherwise. All numbers are our own measurements unless marked otherwise, and Appendix B lists the run files.

7.2 Single facts and small sets (test sets L1 to L4, GB10 and Mac).

This test teaches single facts and small sets into the pocket and asks about them afterwards. On the Mac, a single fact was learned in 218 s. It was recalled 3 of 3 times. Zeroing its neurons gave back the original model word for word in 24 of 24 answers. On the GB10, teaching the same fact failed 4 times in 6 tries. The model also said a confident yes to values it was never taught, did not retire take-backs, and confused facts that look alike. The control questions held at 5 of 5 in every phase.

7.3 Filling the pocket (test set L7, GB10). This test teaches 64 different facts in a row, and after every fifth lesson asks about every fact so far. Of the 64 lessons, 40 were learned, 20 were missed and 4 were later replaced, and the pocket never ran out of room. At the end the model answered 35 of the 64 questions right with thinking off and 29 with thinking on. Most of the loss is facts never learned, not facts forgotten.

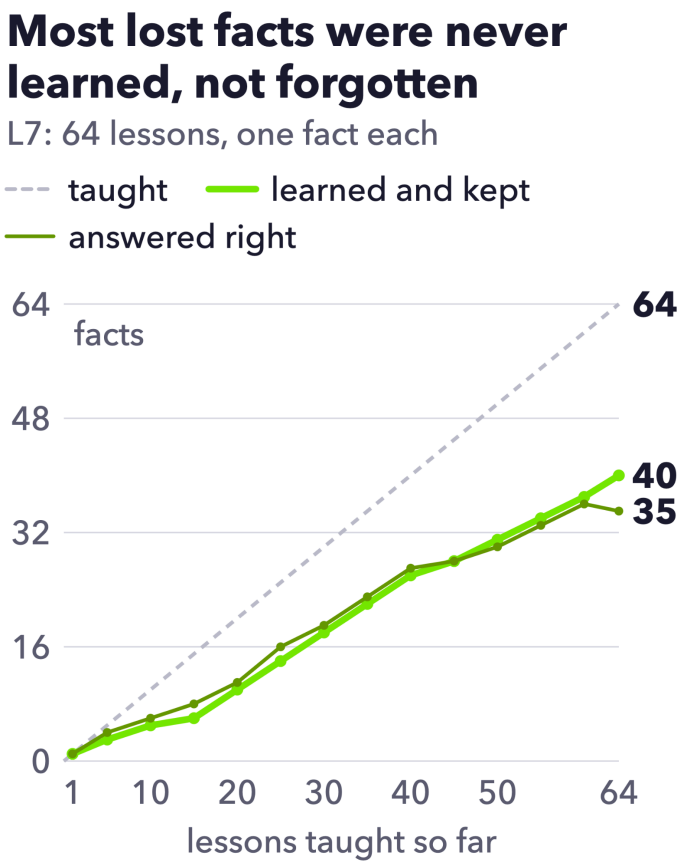


Figure 8. Filling the pocket with 64 different facts about 27 people, pets and things. 40 facts are kept at the end. The model answers 35 of their questions right with thinking off, and 29 with thinking on. Test set L7. GB10, TensorFold 1.0.8, thinking off.

7.4 Two-step questions and the references (test set L6, GB10). This test compares the neurons with the notes and both references on test set L6, which includes two-step questions such as "Which country does Liisa live in?". The model was taught 29 facts. It learned 18, missed 9, and 2 were later replaced. It was asked 56 questions. Each cell gives the result with thinking off and on, and the All row counts each question once.

Question group	Living Weights (thinking off / on)	TensorFold Notes only (thinking off / on)	All lessons in prompt (thinking off / on)	Exact lookup, no model
Current values	8 / 7 of 14	12 / 14 of 14	12 / 13 of 14	14 of 14
Two-step	3 / 5 of 24	16 / 19 of 24	16 / 19 of 24	4 of 24
And, or, not	0 / 2 of 8	5 / 5 of 8	5 / 5 of 8	3 of 8
Yes/no	3 / 1 of 9	7 / 7 of 9	7 / 9 of 9	8 of 9
Negation	0 / 0 of 1	1 / 1 of 1	1 / 1 of 1	1 of 1
Controls	8 / 9 of 9	8 / 8 of 9	8 / 8 of 9	3 of 9

All (each question once)	**22 / 22 of 56**	**43 / 48 of 56**	**43 / 49 of 56**	**29 of 56**

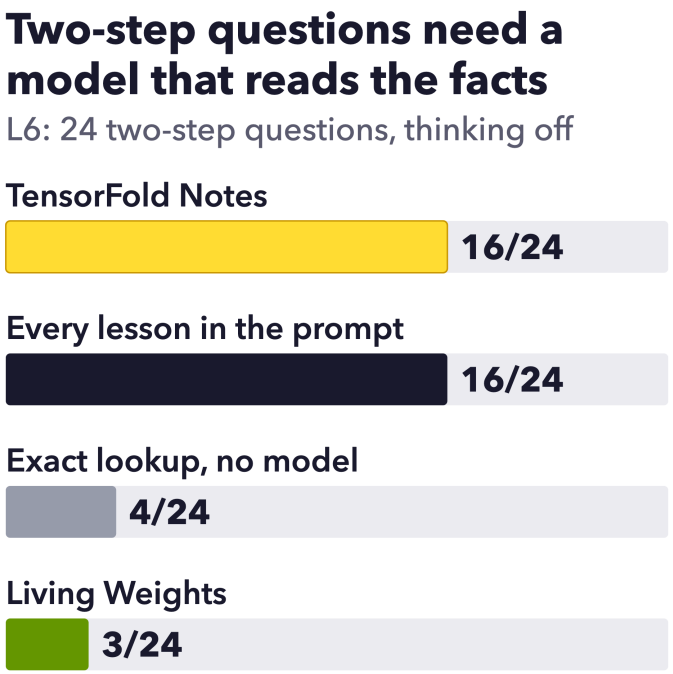


Figure 9. The two-step questions of test set L6, thinking off. L6 is the set with two-step and and/or/not questions (Section 6.2). Notes only and every lesson in the prompt answer 16 of 24. The exact lookup answers 4 and the pocket 3. Every question group is in the table above.

The notes and the full prompt are far ahead of the pocket on everything that needs language, with 43 of 56 against 22 with thinking off. The exact lookup answers every current value and almost every yes/no, and little else. The pocket's two-step score is not yet a chaining score. Of its 10 two-step chains, only 2 had both facts answered right when asked directly. The pocket answered 1 of those 2 as a chain.

7.5 Cost.

This part measures how long learning takes and whether the pocket slows down answering. On the GB10, learning one fact took a median of 179.5 s over 306 lessons. With speculative drafting switched off, a taught pocket decodes at 102.1 tok/s. The untaught model decodes at 102.1 tok/s. An independent check on an RTX PRO 6000 (philip-pentatonic, TensorFold issue #678) agrees within 0.3%. With 64 facts taught, a 4,096-token prompt is 2.0% slower than untaught. The note store takes a median of 0.025 ms per lookup at 1,000 notes and 2.96 ms at 1,000,000 notes.

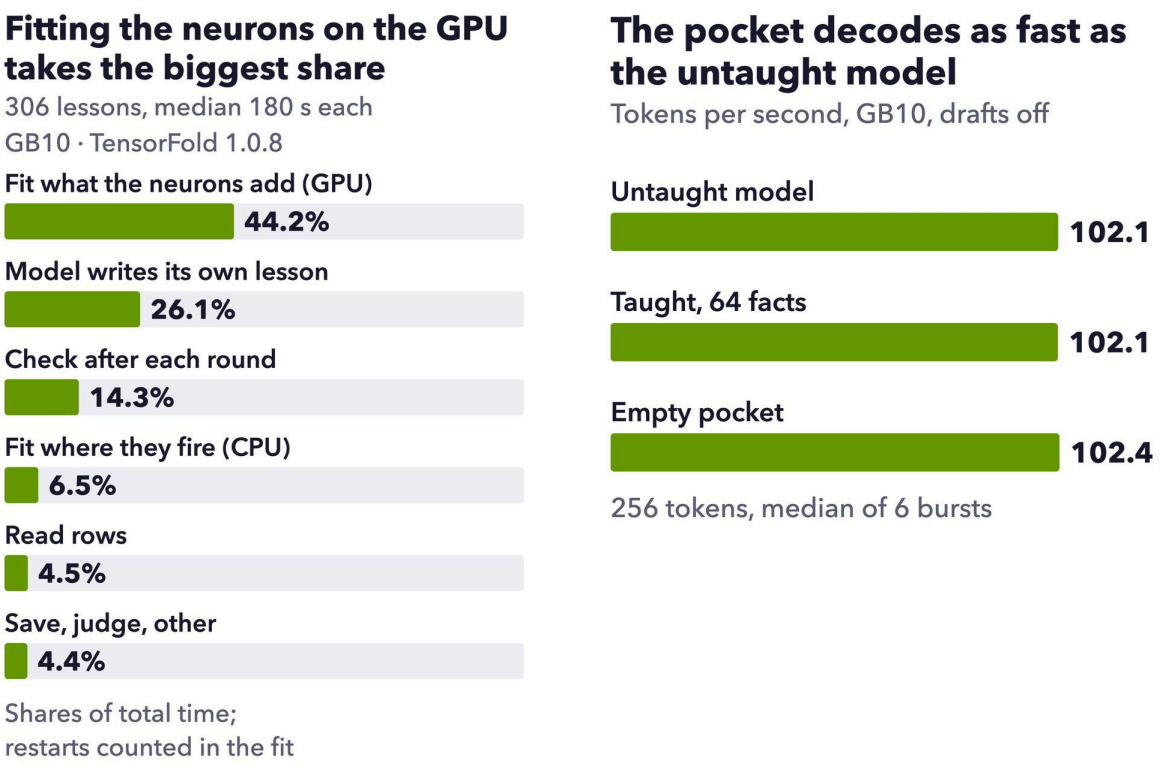


Figure 10. What the neurons cost on a GB10. Left: where one lesson's time goes. There are 306 lessons on TensorFold 1.0.8, with a median of 180 s per lesson. Bars are shares of the total time. Right: decode speed in tokens per second with speculative drafting switched off, for the untaught model, a pocket taught 64 facts and an empty pocket, on the current build (256 tokens, median of 6 bursts).

7.6 Question forms v2: three answer types (test sets L1 to L5, GB10, TensorFold 1.0.8). This test asks every set-up the same questions in newer wordings, with thinking off. It sorts the results by whether the right answer is a value, "no" or "don't know". The pocket is weakest on "no", with 8 of 26 in test set L4. The full prompt is strongest on values and "no". But after take-backs it invents answers where the right one is "don't know". In test set L2 it scores 3 of 17 on these, against 17 of 17 for the exact lookup.

7.7 TensorFold Notes only: the lower layer on the GPU (GB10, TensorFold 1.0.8, notes from our Python prototype). This test gives the untaught model only the notes retrieved for each question, from the prototype of Section 4.7. Each cell gives notes / pocket / all lessons in the prompt, with thinking off. On the small sets L1 to L4 the notes block is longer than all the lessons together, because its fixed framing text outweighs the few notes.

Set	Value	No	Don't know	Prompt tokens, median (TensorFold Notes / all lessons)
L1	12 / 6 / 14 of 15	1 / 1 / 5 of 5	3 / 3 / 3 of 3	91 / 74
L2	4 / 3 / 6 of 6	5 / 1 / 4 of 5	6 / 4 / 3 of 17	99 / 94
L3	28 / 14 / 28 of 34	17 / 8 / 17 of 17	7 / 6 / 5 of 7	127 / 106

L4	32 / 27 / 30 of 36	26 / 8 / 26 of 26	7 / 7 / 7 of 13	194 / 169
L5	37 / 23 / 25 of 50	23 / 11 / 22 of 23	3 / 4 / 3 of 4	130 / 324
L6	35 / 19 / 35 of 41	5 / 0 / 5 of 5	3 / 3 / 3 of 10	351 / 375
Take-back set 1	3 / 3 / 3 of 3	3 / 1 / 3 of 3	3 / 10 / 5 of 13	
Take-back set 2	3 / 3 / 3 of 3	4 / 1 / 4 of 4	9 / 6 / 10 of 16	

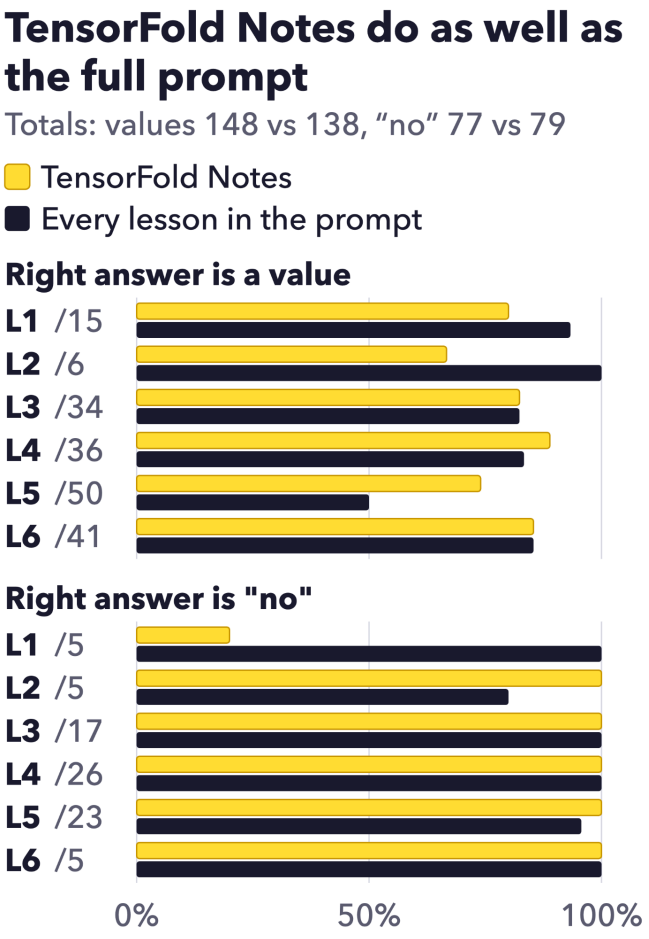


Figure 11. Notes and every lesson in the prompt, per set, when the right answer is a value or “no” (thinking off). In total, notes answer 148 of 182 values and 77 of 81 “no”, the full prompt 138 and 79.

The notes match or beat the full prompt on most sets. They clearly beat the pocket on values and "no". On test set L6, notes answer 43 of 56 with thinking off and 48 with it on. The full

prompt answers 43 and 49. The pocket answers 22. On test set L5 the notes also cut the prompt (130 against 324 tokens) and answer more (37 against 25 of 50). The notes do not yet stop invented answers after a take-back, scoring 3 of 13 on the "don't know" questions of the first take-back set. Adding the taught neurons to the notes does not help. On test set L6 it gives 39 against 43 of 56. This supports the design of Section 5. The notes carry the facts, and promotion is checked against them.

The results above use the notes as they are found and shown today. After these measurements we tried three ways to improve that, on the M5 Max with thinking off. The first lets a name, and the words “I”, “my” and “me”, point to the right person. With the notes of test set L6 mixed into a store of 1,000,000 notes, it raised right answers from 28 to 34 of 56. This change is already in our version of the server, as a setting that is off by default. The second changes the notes block itself: it marks facts that were taken back, and says plainly when the store knows nothing about what was asked. On test set L6 and the two take-back sets it answered 62 of 86 memory questions right. Today’s block answered 43. And it used a prompt a third of the size. The third extends the lookup to questions asked in German and Finnish. At 1,000,000 notes, the German questions that found every note they needed rose from 11 to 24 of 45, and the Finnish ones from 16 to 20. The second and third changes are still prototypes outside the server.

7.8 Promotion: moving facts from the notes into the neurons (test set L6, GB10, thinking off, one run). A new fact is first saved as a note, so the very next answer can use it. Once a night, the server picks the notes that were used in an answer and have not changed since the night before. It teaches these facts to the neurons and then asks a fixed set of other questions. If any of those answers got worse, it undoes the whole night (in this run by zeroing the night's new neurons, Section 5.5). Once the model answers a fact right with that fact’s own note left out, on two nights in a row, the server stops putting the note in the prompt.

How well it works. In our longest run so far, the server taught 15 facts to the neurons in its first six nights. The notes of 5 of them left the prompt and stayed out. Earlier, shorter runs with other settings moved fewer facts or none. We checked that the neurons really know these 5 facts. We asked about them in a separate test with no notes in the prompt at all. As a control, we asked an untaught copy of the same model. The taught model answered all 14 questions right. The untaught copy answered none, saying it had no personal information about the user. Of all 15 taught facts, the neurons answer 12 with no notes.

What it does not do yet. The notes still do most of the work. On the full set of 56 questions, the model answers 36 with its notes and 22 without them. Teaching can also spill over: after learning that the user’s brother has amber eyes, the model started to say amber for other people’s and pets’ eyes too. An earlier run found a bug in the learner: lessons it gave up on still changed the lessons after them and cost answers. Our first fix to the learner removed that damage when the same night was run again. All of this comes from one run on one machine.

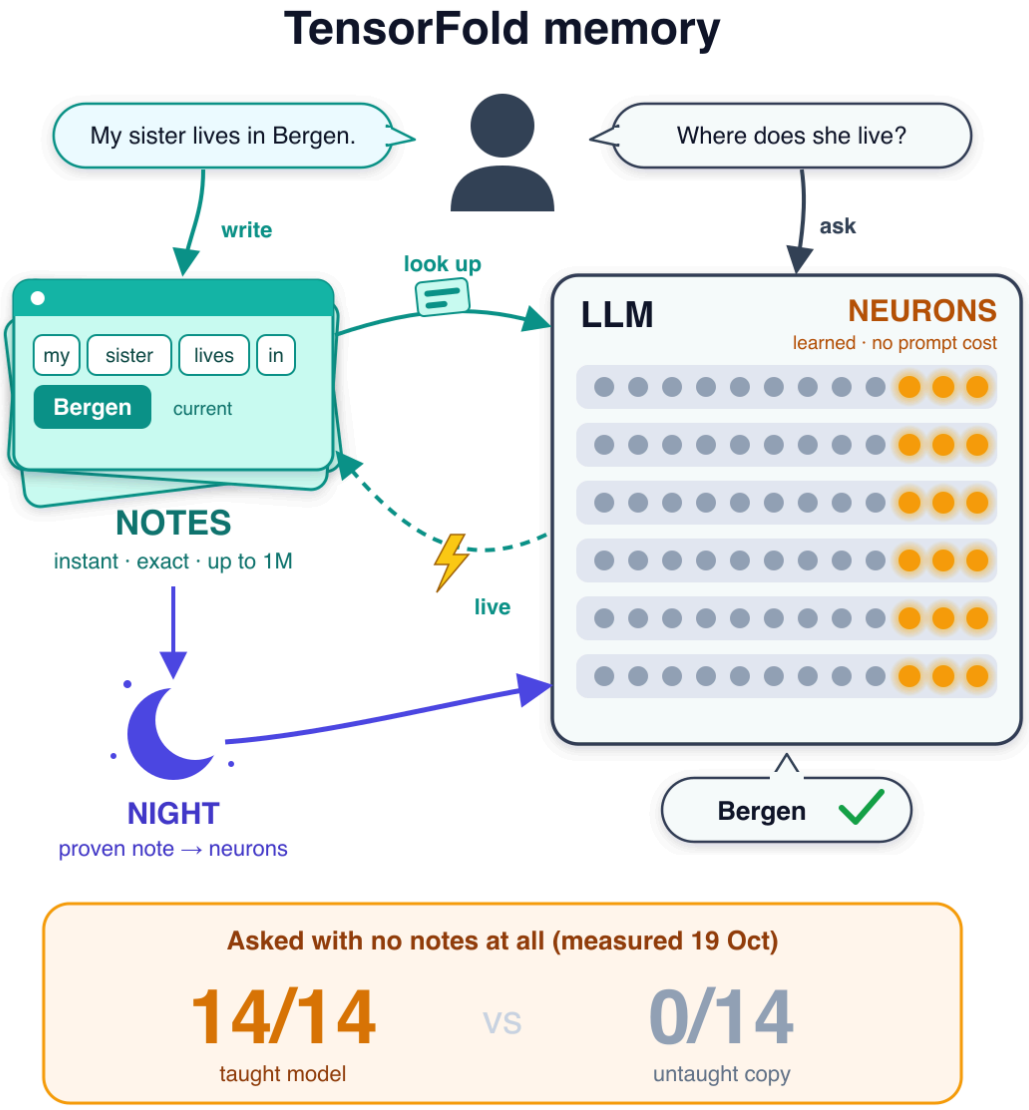


Figure 12. The whole memory in one picture. A new fact is written as a note at once (write). A question brings its notes into the prompt before the answer starts (look up), and more are fetched while the answer is being written (live). Each night, notes that keep being answered right are taught to the neurons (night), which then answer without using prompt tokens. Bottom: asked with no notes at all, the taught model answered the five promoted facts on 14 of 14 questions. An untaught copy answered none (Section 7.8, GB10, one run).

8. Discussion

Why two layers. An exact latest-value table with no model is the test's stand-in for a perfect store. It answers every current value and little else. It gets 14 of 14 current, and 4 of 24 two-step in L6. A model reading every fact as text does far better (43 of 56) and pays prompt tokens on every answer. The first measurement of real notes (Section 7.7) matches the full prompt. Both get the same 43 of 56 on L6. On L5 the notes get more right with a shorter prompt. The pocket answers 22. Notes cannot match a question that shares no word with its note. And they do not yet stop invented answers after a take-back. The first is the neurons' job. The second is the next fix to the notes. The neurons cost no prompt tokens. On the GB10 they cost almost no decode time either. They learn about 6 facts in 10 and answer

stale values after a fact changes. They say yes to values never taught, and do not delete cleanly. Notes give exactness, instant writes and exact deletion from the store. Neurons give token-free answers and reach beyond the stored words. The design puts each where it is strong.

Why promotion is worth the trouble. Prompt cost grows with the number of retrieved notes on every answer, and with many conversations at once it competes for the same prefill compute. A fact asked about every day is cheaper in the weights. A fact that changes weekly is cheaper as a note. Promotion with hysteresis is the rule that decides which is which.

What we cannot claim yet. Three things. First, that notes and neurons together answer more than notes alone: on L5 and L6 they do not (Section 7.7). Second, that promotion keeps every other answer the same over many nights. In our longest run five notes left the prompt. But one taught fact spread to other people’s answers (Section 7.8). Third, that the note store stays cheap beyond the 1,000,000 notes we measured. The first has an experiment in Section 6.4.

9. Limitations

Our results are narrow in three ways. First, every test uses a single model, Nemotron 3.5 Lightning, on two machines, so we do not yet know how other models behave. Second, most test sets were run with only one seed, the number that fixes the random choices made during learning. Another seed could give somewhat different counts, so a difference of one or two questions out of 56 between two set-ups should not be read as real. Third, the learning results were measured on TensorFold 1.0.8. Releases 1.0.9 to 1.0.11 did not change the learning code, and when we repeated three lessons on 1.0.9, the GPU training steps of each lesson took the same time as on 1.0.8 to within 1 s, and the CPU fit to within 1.6 s. A later release could still change the results, and the learner fix tested in Section 7.8 is our own change, not part of any release.

Answers are graded by a program, not by people. The program compares each answer with the expected one using fixed rules. For example, it checks whether the right value appears. It also checks whether the answer says yes, no or "I don't know". When the rules cannot decide, the program marks the answer "unclear" and a person reads it. Rules like these can be wrong. While writing this paper we found and fixed some of their mistakes once, and the fixes changed the grade of 51 of 1,470 answers. Every changed grade is recorded with the run files (Appendix B), so a reader can check each one. With its thinking switched on, the model says "I don't know" more often, so we report those answers separately instead of adding them to the answers with thinking off.

The newest results have limits of their own. When several facts of the same kind are taught, such as the eye colours of different people, a taught value can spread to the wrong person. Answers that need two notes together, such as "Does my cat have the same eye colour as me?", can also break, even when both notes are in the prompt. The documentation of Living Weights lists this interference as a known issue. Our nightly check catches it only by refusing the whole night: with wide checks, an earlier run refused every night and promoted nothing. Our longest run used narrow checks, and 15 facts were promoted. But the checks did not ask the questions where the brother’s amber eyes spread to others (Section 7.8). A first fix

to the learner removed this damage when one such night was repeated, but that is a single run.

The note store has two weak spots. It can find a person by name. But in a large store many people share a name. In our store of 1,000,000 notes, each test name belongs to 500 or more people. So the name alone cannot pick out the right one. And the notes block that says "I don't know", like the lookup in other languages, is still a prototype. Both were measured only on generated questions in English, German and Finnish. Some Finnish words change their consonants when inflected, such as Turku becoming Turussa. These are not handled yet. The system also does not yet detect which language a question is in.

10. Safety and privacy

Whoever can write memory can steer the model: notes are framed as quoted data, and stored-injection tests are part of the suite. Today each server holds one owner’s notes. So they never reach another user’s prompt or training. With several users on one server, notes must be filtered by owner before retrieval. Deletion from the note store is exact. The server's prompt cache and saved conversations are not yet covered. Deletion from neurons is not yet exact. So personal or sensitive facts should stay as notes. And a request to forget must reach the notes, the logs and the backups. A writable table can carry a backdoor [BadEngram 2026]. The promotion gate is also a security gate.

11. Conclusion

A deployed model needs a memory that is fast to write and exact to change, and one that generalises and costs little per answer. We have here built both layers, TensorFold Notes with n-gram stores tested up to 1M facts, and Living Weights for up to 64 facts learned by neurons without additional decode cost on the GB10. The layers integrate in two ways:

First, during inference, relevant Notes are prefetched as input for the LLM in prefill. Then, during answering, new subjects may show up in the answer generated using Living Weights. Notes related to them are live fetched for added knowledge. In other words, the two layers co-operate in creating the answer by reading the facts the other layer brought up and adding to them.

Second, the most useful and durable n-gram Notes are promoted to Living Weights neurons every night. This means that there is a wide, constantly updating storage of facts that improves answers. The LLM learns and generalizes the most important and durable of these facts. They become an integral part of every answering process. They take no room in the prompt. Learning costs time (a median of 180 s per fact on a GB10, done at night); once taught, the model decodes as fast as the untaught one on the GB10 and reads a long prompt about 2% slower.

In our testing, we found that exact notes answer as many questions that need two facts as a prompt that holds every fact. On a mixed set about several people and things, they answer more, and the prompt is shorter. The taught neurons on their own answer facts with no

prompt tokens. When the matching notes are already in the prompt, they do not yet add answers, so their value lies in replacing notes, which promotion is built to do.

Comparing our approach with earlier work, the main difference is where and how fast a fact enters. Weight editors take from a fraction of a second to tens of seconds per fact, and with some editors the accuracy of old memories decays as new edits appear. Retrieval methods leave the weights alone but run beside the model, and some, such as Infini-gram, build their index offline. TensorFold Notes take a new sentence in milliseconds inside the serving engine, while it answers, and change or remove the stored note exactly.

Earlier ways to add learned memory have one of two costs. Some slow every answer: GRACE by a factor of 1.32, WISE by about 3%. Others need a model trained for it from the start: User as Engram, EngramEdit and memory layers. Living Weights learns into the tested Nemotron checkpoint while serving, and with our fix it decodes as fast as the untaught model. These comparisons cross different models and hardware, so they show orders of magnitude, not a ranking.

A last scene from the family illustrates the rule Section 5 proposes. Aunt Liisa tells her notes that she has moved from Tampere to Lapland, in the far north of Finland. The notes have it at once. The neurons were taught Tampere weeks earlier. They still say Tampere. Pekka asks the two-layer model where Liisa lives. The notes answer the change first, as Section 5 proposes, so the model says Lapland, and Pekka drives the green Saab north and finds Liisa at her new door. Had he asked a neurons-only model, he would have driven to Tampere and found an empty flat.



Acknowledgements

The CUDA pocket fix follows two suggestions by Philip Mossop (philip-pentatonic) on TensorFold issue #678. AMD measurements by BlivionIaG.

References

- [Berges 2024] Berges et al., Memory Layers at Scale. arXiv:2412.09764.
- [BadEngram 2026] Fogel, Hofman, Cohen and Vainshtein, BadEngram: Backdoor Attack on Gated Memory Components in LLMs. arXiv:2609.13478 (preprint).
- [Cai 2026] Cai et al., EngramEdit: Decoupled Knowledge Updates in LLMs through Conditional Memory. arXiv:2610.10533.
- [Cheng 2026] Cheng et al., Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models (DeepSeek Engram). arXiv:2601.07372.
- [Fang 2025] Fang et al., AlphaEdit: Null-Space Constrained Knowledge Editing for Language Models. ICLR 2025. arXiv:2410.02355.
- [Geva 2021] Geva et al., Transformer Feed-Forward Layers Are Key-Value Memories. EMNLP 2021. arXiv:2012.14913.
- [Hartvigsen 2023] Hartvigsen et al., Aging with GRACE: Lifelong Model Editing with Discrete Key-Value Adaptors. NeurIPS 2023. arXiv:2211.11031.
- [Huang 2023] Huang et al., Transformer-Patcher: One Mistake worth One Neuron. ICLR 2023. arXiv:2301.09785.
- [Khandelwal 2020] Khandelwal et al., Generalization through Memorization: Nearest Neighbor Language Models. ICLR 2020. arXiv:1911.00172.
- [Kuhn 1990] Kuhn and De Mori, A Cache-Based Natural Language Model for Speech Recognition. IEEE TPAMI 12(6):570–583, 1990.
- [Lample 2019] Lample et al., Large Memory Layers with Product Keys. NeurIPS 2019. arXiv:1907.05242.
- [Li 2026] Li, User as Engram: Internalizing Per-User Memory as Local Parametric Edits. arXiv:2606.19172.
- [Lin 2025] Lin et al., Continual Learning via Sparse Memory Finetuning. arXiv:2510.15103.
- [Liu 2024] Liu et al., Infini-gram: Scaling Unbounded n-gram Language Models to a Trillion Tokens. arXiv:2401.17377.
- [McClelland 1995] McClelland, McNaughton and O'Reilly, Why There Are Complementary Learning Systems in the Hippocampus and Neocortex. Psychological Review 102(3), 1995. doi:10.1037/0033-295X.102.3.419.
- [Meng 2022a] Meng et al., Locating and Editing Factual Associations in GPT. NeurIPS 2022. arXiv:2202.05262.
- [Meng 2022b] Meng et al., Mass-Editing Memory in a Transformer. ICLR 2023. arXiv:2210.07229.
- [Mitchell 2022] Mitchell et al., Memory-Based Model Editing at Scale. ICML 2022. arXiv:2206.06520.
- [Ovadia 2024] Ovadia et al., Fine-Tuning or Retrieval? Comparing Knowledge Injection in LLMs. EMNLP 2024. arXiv:2312.05934.

- [Robertson 2009] Robertson and Zaragoza, The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval 4(1–2):1–174, 2009.
- [TensorFold 2026] Hart, TensorFold: Living Weights guide (docs/living-weights.md) and CHANGELOG, releases 1.0.3 to 1.0.11. github.com/ashhart/TensorFold.
- [TF-Engram 2026] Ma, Huang, Jiang and Shao, TF-Engram: A Train-Free Engram with SSD-Backed Memory for Large Language Models. arXiv:2607.07388.
- [Wang 2024] Wang et al., WISE: Rethinking the Knowledge Memory for Lifelong Model Editing of Large Language Models. NeurIPS 2024. arXiv:2405.14768.
- [Wang 2024b] Wang et al., MEMoE: Enhancing Model Editing with Mixture of Experts Adaptors. arXiv:2405.19086.
- [Wang 2025] Wang et al., MEMOIR: Lifelong Model Editing with Minimal Overwrite and Informed Retention for LLMs. NeurIPS 2025. arXiv:2506.07899.
- [Wei 2026] Wei et al., Deep Fusion Memory: A Chunk-Level Parametric Memory for Large Language Models. arXiv:2610.25100.
- [When to Adapt 2026] Hou and Wang, When to Adapt: Conditional Memory Adapters for Retention-Preserving Domain Specialization. Findings of EMNLP 2026. arXiv:2608.29327.
- [Wu 2022] Wu et al., Memorizing Transformers. ICLR 2022. arXiv:2203.08913.
- [Yang 2025] Yang et al., The Mirage of Model Editing: Revisiting Evaluation in the Wild. ACL 2025. arXiv:2502.11177.
- [Zhang 2024] Zhang et al., A Comprehensive Study of Knowledge Editing for Large Language Models (KnowEdit). arXiv:2401.01286.
- [Zhong 2023] Zhong et al., MQuAKE: Assessing Knowledge Editing in Language Models via Multi-Hop Questions. EMNLP 2023. arXiv:2305.14795.

Appendix A. The rule book in one page

We teach the model short facts, one lesson at a time. Each lesson is about one entity and one attribute, such as “my eyes” or “my cat’s name”. There are four kinds of lesson. Teaching states a value, either now or over a dated interval (“since 3 March 2025”, “until 11 October 2026”, “always”). A change says the value is new now, and it keeps the old value as history. Taking back says a value no longer holds, or that no value holds. A dated change says the value was one thing until a date and another from that date on. Any lesson can later be deleted.

For each (entity, attribute) pair, the rule book keeps one truth value per possible value at every moment: yes, no or unknown. Anything never said is unknown, never no. To find the state at a moment, the rule book replays the lessons still in force in the order they were taught; where two lessons cover the same time, the later one wins. Some wordings also carry a hint: “not blue anymore” implies it was blue just before. A hint only fills a gap and never overrides what a lesson says outright. Most attributes hold one value at a time, but likes are a set.

The right answer follows from that state. A question about the value now, on a date, or just before a date gets the value marked yes; it gets “none” if every value is no, and otherwise “don’t know”, noting any values ruled out. A yes/no question reads the value’s truth directly. An either-or question gets the option that is yes, “neither” if both are no, and otherwise “don’t know”. A question about the past value gets the last known value before the current one. “Has it always been the same?” is yes if every piece of the history has the same value, no if

two pieces conflict, and otherwise “don’t know”. “Who has it?” lists the entities for which the value is yes. General-knowledge questions keep their textbook answer.
Two properties matter most. First, deleting a lesson gives exactly the answers of a store where it was never taught, in any delete order. Second, teaching a lesson about one pair changes no answer about any other pair.
The rule book exists twice. The Python copy runs, and all 104 of its self-checks pass, including negative controls on eight systems that are broken on purpose. The Lean copy states the properties as 93 theorems, and Lean 4.35.0 checks every one. The two copies agree on all 5,910 expected answers of the example sets.

Appendix B. Code and data

All paths below are in github.com/ThinkOffApp/dynamic-ngram-memory. It is the TensorFold Notes module as a general-use repository, under the Apache-2.0 license. It is a pre-release snapshot that runs inside the pinned TensorFold host. It also holds our test suite (MIT license) and the tables and receipts behind Section 7.

- Test suite: the generated test sets, the rule book in Python and in Lean, the grading program and the runner are in tests/.
- Results for Section 7, with the file each number comes from: docs/PAPER-RESULTS-DETAIL.md.
- Note store measurements (Sections 4.2, 4.7 and 7.7): docs/NOTES-LAYER.md and docs/NOTES-ZIG.md, with raw data in docs/notes-raw/.
- Promotion (Sections 5 and 7.8): docs/PROMOTION-IN-SERVER.md, and two GPU runs with every answer in docs/receipts/: the no-notes control and erasing a promoted fact.
- First fact tests and the corrections to the grading rules (Sections 7.2 and 9): docs/RESULTS.md and docs/rejudge/.
- Raw run files (every answer, its grade, and the server and run logs) are kept with the results in the same repository.